

EatAps

Полный аудит приложения

Дата аудита	11 сентября 2026
Репозиторий	/Users/denyskrikshchiunas/Projects/EatAps
Ветка	cleanup/dead-is-favorite
Аудируемый коммит	6034515cde858d7c83a20bcd20a0092837d5a91
Версия приложения	0.1.0
Объём кода	~75 800 строк · 73 компонента · 64 модуля · 12 535 строк SQL

Итог

0	3	5	12	2
P0	P1	P2	P3	P4

Всего находок	22
Готовность к продакшену	66 / 100
Покрытие аудитом	95.3 % (513 элементов)
Сборка	npm run build — успешно
Тесты	813 из 814 пройдено, 1 провален (sqlGuard)

ТРЕБУЮТСЯ СУЩЕСТВЕННЫЕ ИСПРАВЛЕНИЯ

Инженерно приложение зрелое. К выпуску не готовы правовые документы и три пункта P1.

Аудит выполнен статическим разбором, запуском сборки и тестов, а также прогоном расчётных модулей на граничных входных данных. Браузерный и боевой прогон не проводился — см. раздел «Границы аудита».

Содержание

1. Резюме для владельца	4
1.1. Что сделано хорошо	4
1.2. Что мешает выпуску	5
1.3. Оценка	5
2. Границы аудита — что проверено и что нет	7
2.1. Что выполнено фактически	7
2.2. Чего аудит НЕ делал	7
2.3. Как читать степень уверенности	8
3. Архитектура	9
3.1. Общее устройство	9
3.2. Три решения, на которых держится проект	9
3.3. Значения по умолчанию	10
4. Инвентарь приложения	11
4.1. Числа	11
4.2. Распределение интерактивных элементов	11
5. Сборка, тесты и оснастка	13
5.1. Результаты запусков	13
5.2. Оснастка проекта — сильная сторона	13
5.3. Пробелы в покрытии тестами	13
6. Аудит по экранам	15
7. Аудит взаимодействий	18
7.1. Состояния загрузки, пустоты и ошибки	19
8. Аудит серверных точек	20
8.1. Общие свойства серверного слоя	20
9. Аудит базы данных и RLS	21
9.1. Сводка	21
9.2. Предикаты доступа — разбор	21
9.3. Проверка на подмену личности и IDOR	22
9.4. Хранилище файлов	22
10. Модель приватности — ожидаемое против фактического	23
11. Инварианты системы	24
12. Офлайн, многоустройство и конфликты	25
12.1. Механика	25
12.2. Разбор сценария	25
12.3. Единственная оговорка	26
13. PWA, service worker и обновление	27
13.1. Приватность кэша — проверено отдельно	27
13.2. Обновление версии	27
14. Аудит AI-ассистента	28
14.1. Поток и разграничение прав	28
14.2. Медицинские границы	28
15. Производительность и качество кода	30

15.1. Проверено	30
15.2. Замечания к архитектуре кода	30
16. Доступность и мобильный интерфейс	31
17. Приватность и соответствие GDPR	32
17.1. Что делается правильно	32
17.2. Что не соответствует	32
18. Находки	33
SEC-001 — Дневной лимит AI обходится параллельными запросами: проверка и списание не атомарны	34
PRIV-001 — Anthropic, Stripe, Telegram и Open Food Facts не раскрыты ни в политике, ни в реестре обработ...	35
PRIV-002 — Загруженные изображения не удаляются никогда — в том числе при удалении поста и всего ак...	37
SEC-002 — Незаданная переменная цены превращает неизвестный прайс в высший тариф AI_PREMIUM	39
SEC-003 — /api/feedback открыт без входа, защищён только заголовком Origin и счётчиком в памяти процес...	40
DB-001 — setup_all.sql правлен руками, новая миграция не зарегистрирована в сборке; следующая перес...	42
SEC-004 — Вебхук Stripe не защищён от переупорядоченной доставки: позднее старое событие воскрешае...	43
PRIV-003 — Выгрузка данных не включает переписку, мысли, комментарии, реакции, события и социальны...	44
UX-001 — Количество продукта не ограничено сверху: 9999999999999999 г принимаются и портят всю статисти...	45
BUG-001 — computeTargets не проверяет вход: нечисловой профиль даёт NaN во всех целях, Infinity в весе...	47
SEC-005 — Блокировка не распространяется на общие групповые диалоги	48
DEP-001 — Неиспользуемый стек Web3 приносит 12 из 17 уязвимостей npm и чанк на 2,68 МБ в каждой сб...	49
A11Y-001 — Кнопки-символы «←» и «→» не имеют доступного имени	51
SEC-006 — CSP разрешает unsafe-inline и unsafe-eval, connect-src открыт на любой https	52
SEC-007 — Тип загружаемой в AI картинки проверяется только по заявленному media_type, без сигнатуры...	53
UX-002 — Оптимистичные переключатели приватности могут рассинхронизироваться с сервером при быс...	54
CFG-001 — Личные Telegram chat ID записаны в исходном коде и продублированы как значение по умолч...	55
DB-002 — Восемь триггерных функций объявлены без set search_path	56
UX-003 — Поиск по префиксу позволяет перебрать базу пользователей; ограничения частоты на поиск нет	58
AI-001 — Границы медицинской безопасности заданы только промптом; серверной проверки ответа нет	59
CODE-001 — src/lib/exchange.js — 311 строк экспорта и импорта CSV, не подключённых ни к одному экрану	60
PERF-001 — Главный чанк 1,15 МБ (341 КБ в gzip); разбиение на части не настроено	61
19. Десять самых значимых проблем	63
20. Двадцать быстрых исправлений	64
21. План устранения	65
Фаза 0 — до любого публичного доступа	65
Фаза 1 — до открытия для широкой публики в ЕС	65
Фаза 2 — следующий выпуск	65
Фаза 3 — качество и архитектура	65
22. Покрытие аудита	66
23. Готовность к продакшену	67
24. Можно ли выпускать EatAps сегодня?	68
Почему нет	68
Почему это всё равно хорошая новость	68
Что обязательно сделать до запуска	68
25. Приложение	70
25.1. Состав аудита	70
25.2. Выходные файлы	70
25.3. Как воспроизвести проверки	70
25.4. Оговорка о полноте	70

1. Резюме для владельца

EatAps — трекер питания в виде PWA: дневник еды с расчётом КБЖУ и микронутриентов, синхронизация между устройствами с работой офлайн, социальная часть (подписки, закрытые аккаунты, лента, личные и групповые сообщения), AI-ассистент на моделях Anthropic, подписки через Stripe и модерация через Telegram-бота.

Первое, что нужно сказать прямо: это не типовый проект, собранный из шаблонов. Инженерная культура здесь заметно выше средней. Ряд вещей, которые в аудитах обычно составляют список претензий, здесь сделаны правильно ещё до аудита.

1.1. Что сделано хорошо

- Row Level Security включён на BCEX 33 таблицах без исключений, а таблица промокодов намеренно оставлена вовсе без политик — то есть закрыта по умолчанию. Это встречается редко.
- Все функции с правами определившего (SECURITY DEFINER) объявлены с `set search_path = public`. Проверка сплошная, по всем 148 функциям.
- Подмены личности нет нигде: реакции, сообщения, подписки и настройки везде опираются на `auth.uid()`, а не на идентификатор из запроса. Отдельно проверялись самые частые места таких ошибок — ни одного совпадения.
- Уведомления невозможно подделать: политики INSERT у таблицы нет вовсе, а функция `push_notification` отозвана у роли `authenticated`.
- Гашение промокодов защищено правильно и на двух уровнях сразу: `SELECT ... FOR UPDATE` в функции и ограничение `CHECK (used <= max_uses)` в самой таблице. Гонка «сто человек на последнее место» здесь невозможна.
- Синхронизация между устройствами — сильнейшая часть проекта: гибридные логические часы, тумбстоуны на каждое удаление, отдельные метки времени на каждое поле дня, кэш с ключом по пользователю (перепутать аккаунты физически нечем) и атомарный `compare-and-swap` по `revision` в `save_app_state`.
- Service worker не кэширует ничего приватного: чужие `origin` (то есть весь Supabase) и `/api/` пропускаются мимо кэша, ссылки входа тоже. Приватные данные следующему пользователю устройства не достанутся.
- Из 80 проверенных `useEffect`, ставящих слушатели, таймеры или подписки, ровно ноль оставлены без функции очистки.
- XSS-поверхности нет: ни `dangerouslySetInnerHTML`, ни `innerHTML`, ни `eval` во всём проекте; автолинковка в чате разбирает только `http(s)`.
- Ограничения частоты на изменяющие действия реализованы в базе триггерами, а не в интерфейсе: подписки, публикации, ответы, реакции, сообщения незнакомым.
- Сканер штрихкода обрабатывает все шесть ветвей отказа (нет разрешения, нет камеры, камера занята, небезопасный контекст, не поддерживается, прочий сбой), и в каждой даёт выход «Добавить вручную».
- Собственная оснастка вокруг SQL: сборка `setup_all.sql` из источников, проверка идемпотентности, аудит политик, каталог схемы — и всё это под автотестами.
- 814 автотестов, включая сценарии на два устройства, слияние состояний и защиту от воскрешения удалённых записей.
- Комментарии в коде объясняют ПРИЧИНУ решения и то, что сломалось раньше, а не пересказывают код. Это заметно упрощает сопровождение.

1.2. Что мешает выпуску

Найдено 22 подтверждённых расхождения. Критических (P0) нет: ни захвата аккаунта, ни массовой утечки приватных данных, ни обхода оплаты в полном смысле обнаружить не удалось. Но три пункта уровня P1 выпускать в таком виде нельзя.

Если читать только одну строку

Главная проблема проекта — не в коде, а в документах. Правовые тексты описывают приложение, каким оно было до появления AI-ассистента, подписок и поддержки, и показываются людям с красной плашкой «Шаблон» и незаполненными полями вида [Name], [Anschrift]. Для приложения с немецкими юридическими текстами, обрабатывающего данные о здоровье, это блокирующий пункт.

SEC-001 — Дневной лимит AI обходится параллельными запросами: проверка и списание не атомарны

Прямой денежный ущерб владельцу ключа Anthropic: платит он, лимит не действует. ~33-кратное превышение при 100 потоках, повторяемо ежедневно и с любого числа бесплатных аккаунтов. Одновременно обесценивается платная подписка: тариф FREE даёт то, за что берут деньги.

PRIV-001 — Anthropic, Stripe, Telegram и Open Food Facts не раскрыты ни в политике, ни в реестре обработки; политика опубликована как незаполненный шаблон

Для приложения с немецкими правовыми текстами (то есть нацеленного на рынок ЕС), обрабатывающего данные по ст. 9, это нарушение ст. 13, 30 и, при отсутствии договоров, ст. 28 GDPR. Отсутствие названного ответственного лица и контакта делает невозможной реализацию прав субъекта — то самое, что проверяется в первую очередь по жалобе. Это блокирующий пункт для публичного запуска в ЕС, а не косметика.

PRIV-002 — Загруженные изображения не удаляются никогда — в том числе при удалении поста и всего аккаунта; бакеты открыты на чтение

Нарушение права на удаление (ст. 17 GDPR): после удаления аккаунта персональные данные — фотографии, в том числе с людьми и обстановкой, — продолжают отдаваться с сервера проекта. Дополнительно: фото из «закрытых» мыслей не защищены ничем, кроме неразглашения ссылки, и бесконечный рост платного хранилища.

1.3. Оценка

Направление	Балл	Чем обоснован
Функциональная корректность	8/10	Ядро работает, 813 из 814 тестов зелёные. Снимают балл незакрытая верхняя граница количества и NaN при повреждённом профиле.
Безопасность	6/10	RLS сплошной и грамотный, подмены личности нет нигде. Снимают баллы гонка бюджета AI и отказ в высший тариф.
Приватность	4/10	Модель доступа в базе образцовая, но изображения не удаляются никогда, а правовые документы не знают о трёх обработчиках.
Целостность базы	8/10	RLS на всех 33 таблицах, каскады полные, save_app_state атомарен. Снимает балл разъехавшийся setup_all.sql.
Офлайн и синхронизация	9/10	HLC, тумбстоуны, пофайловые метки, кэш по пользователю, 814 тестов вокруг слияния. Сильнейшая часть проекта.
Платежи	6/10	Подпись вебхука проверяется, клиент тариф не назначает. Снимают баллы отказ в AI_PREMIUM и отсутствие контроля порядка событий.
Безопасность AI	6/10	Границы в промпте продуманы, тариф серверный. Снимают баллы обход лимита и отсутствие второго рубежа.

Направление	Балл	Чем обоснован
Производительность	7/10	Ни одной утечки эффектов на 80 проверенных. Снимает балл неразделенный чанк на 1,15 МБ.
Пользовательский опыт	8/10	Отмена, подтверждения, честные пустые состояния, все ветки отказа сканера. Снимает балл рассинхронизация тумблеров приватности.
Доступность	6/10	126 aria-label и корректные роли, но десять кнопок-символов без имени на главном экране.
Покрывтие тестами	8/10	814 тестов, включая многоустройственные сценарии и сторожей SQL. Не покрыты RLS, платежи и лимиты AI.
Сопровождаемость	8/10	Комментарии объясняют причины, а не механику. Снимают балл мёртвый Web3 и exchange.js.
Наблюдаемость	4/10	Есть структурный log и console.error в критичных местах, но ни сбора ошибок, ни метрик, ни оповещений.
Готовность к продакшену	5/10	Инженерно приложение зрелое; к выпуску не готовы правовые документы и три пункта P1.

ИТОГОВАЯ ОЦЕНКА: 93 из 140 → 66 / 100

Вердикт

ТРЕБУЮТСЯ СУЩЕСТВЕННЫЕ ИСПРАВЛЕНИЯ — до устранения трёх пунктов P1 приложение не следует открывать для широкой публики в ЕС. Для закрытого теста на ограниченном круге согласившихся пользователей состояние приемлемо.

2. Границы аудита — что проверено и что нет

Отчёт разделяет проверенное от предполагаемого. Ниже честно перечислено, чего этот аудит НЕ делал, чтобы ни одна оценка не была принята за большее, чем она есть.

2.1. Что выполнено фактически

- Сплошное чтение кода: 73 компонента, 64 модуля, 8 серверных точек, 12 535 строк SQL, 24 миграции, service worker, конфигурация сборки и развёртывания.
- Запуск сборки: `npm run build` — успешно, 11,12 с, предупреждения о размере чанков зафиксированы.
- Запуск тестов: `npm test` — 813 из 814 пройдено, один провал (sqlGuard) разобран и вошёл в находку DB-001.
- Запуск собственной оснастки проекта: `policy-audit`, `check-sql`, `sql-idempotency`, `sql-catalog`, `build-setup-all --check`.
- Прогон расчётных модулей на граничных данных: `computeTargets`, `scale`, `sanitizeAmount`, `amountLabel`, `formatAmount` — 15 наборов входных данных, включая ноль, отрицательные, `1e12`, `NaN`, `Infinity`, запятую вместо точки, нечисловые строки и нелатинские цифры.
- Проверка семантики `TIER_BY_PRICE` запуском на реальных значениях окружения — находка SEC-002 подтверждена исполнением, а не рассуждением.
- `npm audit` с разбором каждой из 17 записей по дереву зависимостей.
- Автоматический разбор всех 80 `useEffect` на наличие функции очистки.
- Автоматический разбор всех определений функций SQL на `security definer` и `search_path`.

2.2. Чего аудит НЕ делал

Важно

Ни один запрос к боевым адресам (eataps.com, Supabase, Stripe, Telegram, Anthropic) не отправлялся. Данные реальных пользователей не затрагивались. Все сценарии воспроизведения в отчёте описаны, но НЕ исполнены против прода.

- Браузерный прогон интерфейса не проводился: в окружении нет ни Chromium, ни средств автоматизации браузера. Все выводы по экранам и кнопкам получены чтением кода. Поэтому по каждому экрану в матрице покрытия стоит «runtime: нет».
- Живой экземпляр Postgres недоступен, поэтому политики RLS проверены разбором предикатов, а не выполнением запросов от имени разных пользователей. Это существенная оговорка: разбор ловит логические дыры, но не ловит ошибок исполнения.
- Боевое окружение Vercel недоступно: какие переменные там заданы — неизвестно. Выводы о `STRIPE_PRICE_AI_PREMIUM` и `VITE_WALLETCONNECT_PROJECT_ID` сделаны по отсутствию документации в `.env.example` и помечены соответствующим уровнем уверенности.
- Живых прогонов AI-модели не выполнялось: устойчивость к внедрению инструкций и качество медицинских границ оценены по тексту промпта, а не по поведению.
- Вход через Google (OAuth) и через кошелёк Web3 не проверялись сквозным прогоном: первый требует боевых ключей, второй выключен.
- Нагрузочное тестирование, проверка на реальных мобильных устройствах, аудит Lighthouse и проверка контраста пикселями не проводились.

2.3. Как читать степень уверенности

Метка	Что означает
CONFIRMED	Дефект доказан: исполнением кода, выводом инструмента или однозначным чтением. Спорить не о чем — вопрос только в приоритете.
PROBABLE	Путь рассуждения проверен по коду, но сам сценарий не воспроизводился. Требуется подтверждения перед тем, как считать фактом.
HIGH / MEDIUM	Уверенность в оценке ПОСЛЕДСТВИЙ и достижимости, отдельно от факта существования дефекта.

3. Архитектура

3.1. Общее устройство

Приложение построено по принципу «сначала локально»: состояние живёт на устройстве, облако — способ синхронизации, а не источник истины. Отсюда вытекает почти вся его архитектура.

```

Браузер / PWA
├─ React 18 + Vite 6 (без маршрутизатора: экраны — состояние, а не адреса)
├─ store.jsx         единое состояние + 40 мутаций + гибридные логические часы
├─ localStorage      кэш с ключом eataps:state:<uid> — по пользователю
├─ service worker     только статические ресурсы; /api/ и чужие origin не кэширует
├─ syncEngine         очередь отправки, повтор, слияние, статусы
├─
├─ Supabase — Postgres (33 таблицы, RLS на всех)
│   │   148 функций, 110 доступны роли authenticated
│   │   Realtime: состояние, сообщения, события, подписки
│   │   Storage: post-images, chat-images (публичные),
│   │               dm-media (закрытый, подписанные ссылки)
├─
├─ /api/* — Vercel Functions (8 точек)
├─ ai/chat, ai/vision → Anthropic (ключ только на сервере)
├─ stripe/checkout, portal, webhook
├─ telegram/webhook   → модерация
├─ support, feedback  → Telegram владельца

```

3.2. Три решения, на которых держится проект

Состояние как один документ

Весь дневник пользователя — одна строка `app_state` с полем `jsonb` и счётчиком `revision`. Запись идёт через `save_app_state` с условием `where revision = p_base_revision`: это `compare-and-swap`, атомарный по построению. Проигравший гонку получает `out_conflict = true` и текущее состояние сервера, после чего сливает его у себя. Решение нестандартное, но для дневника оправданное и реализовано корректно.

Права считаются по подпискам, а не по отдельной таблице дружбы

Друзья — это взаимная подписка. Таблица `friendships` осталась только якорем для уведомления «теперь вы друзья», а все права считает `is_friend_with` по `follows`. Это закрывает целый класс расхождений, когда право переписки и право видеть дневник разъезжались. В проекте есть даже отдельный тест-сторож, следящий, чтобы политика дневника спрашивала `is_friend_with`, а не читала строку `friendships`.

Приватность многослойная, и слои перекрывают друг друга сверху вниз

Блокировка сильнее всего, затем закрытость аккаунта, затем выбранный владельцем круг, и отдельно — поимённые разрешения (доступ к дневнику, тренер), которые закрытость не отменяет. Предикаты `can_view_diary`, `can_view_post`, `can_view_profile_content` написаны последовательно и согласованно; расхождений между ними при проверке не обнаружено.

3.3. Значения по умолчанию

Отдельно отмечу выбор значений по умолчанию — он консервативный, то есть в пользу приватности. Это редкость и её стоит сохранить.

Настройка	По умолчанию	Оценка
Аккаунт закрыт (is_private)	false	Открытый по умолчанию — обычная практика для социальной части, приемлемо.
Видимость дневника	mutuals	• Верно: самое личное открыто только взаимным подпискам, а не всем.
Сообщения от подписок	direct	Разумно: на кого подписан — тому доверяешь.
Сообщения от подписчиков	request	• Верно: незнакомец попадает в запросы, а не в чаты.
Сообщения от остальных	request	• Верно.
Приглашения в группы	following	• Верно — и именно это ограничивает последствия SEC-005.
Отчёты о прочтении	true	Спорно, но симметрично и отключаемо.

4. Инвентарь приложения

4.1. Числа

Что	Сколько	Примечание
Файлов под контролем версий	282	без node_modules и dist
Строк кода (всего)	~75 800	src + api + supabase + scripts
React-компонентов	73	включая messaging/ и social/
Экранов (монтируются целиком)	35	перечислены в разделе 6
Модулей src/lib	64	без тестовых файлов
Тестовых файлов	49	814 тестов
Интерактивных обработчиков	756	538 onClick, 187 onChange, 8 onSubmit, 23 прочих
Тегов <button>	405	
Полей ввода	105	input, textarea, select
Серверных точек /api	8	
Таблиц Postgres	33	RLS включён на всех 33
Функций Postgres	148	110 доступны роли authenticated
RPC, вызываемых клиентом	68	все имеют соответствующий grant
Политик RLS (последние редакции)	99	113 операторов create policy с учётом истории
Триггеров	33	
Индексов	47	
Миграций	24	одна не внесена в сборку — DB-001

4.2. Распределение интерактивных элементов

Сложность сосредоточена в листе добавления еды: 134 обработчика, 81 кнопка и 46 полей ввода в одном файле на 2 267 строк — это самый нагруженный экран приложения и главный кандидат на разделение (см. раздел 15).

Компонент	Обработчиков	Кнопок	Полей
AddMealSheet.jsx	134	81	46
DayScreen.jsx	48	32	2
SettingsPanels.jsx	40	10	3
SupplementSheet.jsx	29	19	6
ThoughtsFeed.jsx	29	16	3
Onboarding.jsx	25	7	4
ConversationInfo.jsx	19	7	1
ChallengesScreen.jsx	19	10	4
ChatScreen.jsx	18	7	0

Компонент	Обработчиков	Кнопок	Полей
PrivacyHub.jsx	17	1	0
остальные 63 компонента	378	215	36

5. Сборка, тесты и оснастка

5.1. Результаты запусков

Команда	Итог	Что зафиксировано
npm test	ЧАСТИЧНО	813 из 814 пройдено за 1,8 с. Провал: «setup_all.sql собран из источников и не разошёлся с ними» (src/lib/sqlGuard.test.js:37) → находка DB-001.
npm run build	УСПЕХ	Собрано за 11,12 с. Предупреждение rollup о чанках больше 500 КБ → PERF-001. Скрипт inject-precache отработал.
npm run sql:verify	ПРОВАЛ	Падает на первом шаге — тот же DB-001.
node scripts/check-sql.mjs	УСПЕХ	Все SQL-файлы, включая новую миграцию, прошли статическую проверку.
node scripts/sql-idempotency.mjs	УСПЕХ	Нарушений нет на 23 файлах — повторный прогон миграций безопасен.
node scripts/policy-audit.mjs	УСПЕХ	Выведены все политики по таблицам; использован как основа раздела 9.
npm audit	ЗАМЕЧАНИЯ	17 уязвимостей (10 умеренных, 7 высоких) → DEP-001.
npm outdated	ЗАМЕЧАНИЯ	React 18 при доступном 19, Vite 6 при 8, supabase-js и stripe на минор отстают.

5.2. Оснастка проекта — сильная сторона

Проект содержит собственные инструменты контроля SQL, которых обычно нет даже в зрелых кодовых базах: детерминированная сборка файла полной установки, сплошная проверка идемпотентности каждого создаваемого объекта, аудит политик, сверка каталога схемы с миграциями. Всё это вызывается из тестов, то есть нарушение ловится автоматически.

Единственная претензия к этой оснастке — не к ней самой, а к тому, что её сигнал сейчас проигнорирован: тест DB-001 честно упал, но код с расхождением остался в рабочем дереве.

5.3. Пробелы в покрытии тестами

Область	Покрытие	Что нужно добавить
Слияние и синхронизация	отличное	Сценарии на два устройства, тумбстоуны, конфликты — покрыто лучше всего в проекте.
Расчёты питания	хорошее	Добавить граничные входы: пустой профиль, Infinity, отрицательные (см. BUG-001).
Сторожа SQL	отличное	Дополнить проверкой search_path у всех функций (DB-002).
Политики RLS	нет	Главный пробел. Нужны интеграционные тесты на живой базе: два пользователя, блокировка, закрытый аккаунт, круги дневника.
Лимиты AI	частичное	aiBudget покрыт как чистая функция; конкурентный доступ не проверяется вовсе — именно там SEC-001.
Платежи	минимальное	Покрыт только safeOrigin. Нужны тесты на отображение цены в тариф и на порядок событий.

Область	Покрытие	Что нужно добавить
Аутентификация	частичное	authState и authErrors покрыты; сквозных сценариев входа и выхода нет.
Приватность (сквозная)	нет	Нет теста, проверяющего, что скрытое в интерфейсе действительно недоступно через API.

6. Аудит по экранам

Ниже все 35 экранов приложения. Для каждого — файл, способ открыть, основные функции и результат разбора. Напоминаю границу: это статический разбор, браузерного прогона не было (см. раздел 2).

ID	Экран	Файл	Результат разбора
S-0 1	Онбординг	Onboarding.jsx	Проверка ввода на входе сделана правильно: bodyOk требует возраст 10–100, рост 120–230, вес 30–250, кнопка «Продолжить» до этого заблокирована. Сам расчёт при этом защиты не имеет — BUG-001. → BUG-001
S-0 2	День (дневник)	DayScreen.jsx	Отмена добавления вынесена в App и переживает закрытие листа — редкое и правильное решение. Кнопка закрытия календаря без aria-label. → A11Y-001
S-0 3	Лист добавления еды	AddMealSheet.jsx	Самый нагруженный экран: 134 обработчика, 81 кнопка, 46 полей. Количество продукта не ограничено сверху; девять кнопок «назад» без доступного имени. → UX-001,A11Y-001
S-0 4	История	HistoryScreen.jsx	Повтор дня копирует СНИМКИ продуктов с новыми идентификаторами — правка копии не трогает оригинал. Сделано верно.
S-0 5	Статистика	StatsScreen.jsx	Исключение дня из статистики и «учитывать всё равно» версионизируются отдельными метками. Смягчает последствия UX-001.
S-0 6	AI-ассистент	AITab.jsx → AIHomeScreen	Тариф и модель берутся из базы, а не из запроса — подмена невозможна. Но дневной лимит обходится параллельными запросами. → SEC-001,AI-001
S-0 7	Тарифы AI	AIPlansScreen.jsx	Возврат из Stripe опрашивает подписку до восьми раз с паузой — вебхук может опоздать, и это учтено. → SEC-002
S-0 8	Лента	FeedTab.jsx → FeedScreen	Лента строится серверной функцией list_feed; видимость проверяется в базе, а не на клиенте.
S-0 9	Общение (входящие)	messaging/InboxScreen.jsx	Все счётчики непрочитанного приходят одним серверным вызовом unread_totals — раньше клиент считал сам и расходился между устройствами.
S-1 0	Чат	messaging/ChatScreen.jsx	Реакции переключаются на сервере по auth.uid(). Медиа диалогов лежат в закрытом бакете с подписанными ссылками — сделано правильно. Старые фото чата остаются в публичном бакете. → PRIV-002,SEC-005
S-1 1	Новое сообщение	messaging/NewMessageScreen.jsx	Создание группы проверяет блокировку дважды и настройку group_invites приглашаемого.
S-1 2	Запросы на переписку	messaging/MessageRequestsScreen.jsx	Ограничение «не больше 5 сообщений, пока не ответили» и «не больше 20 новых собеседников в час» реализовано триггером в базе.
S-1 3	О диалоге	messaging/ConversationInfo.jsx	Четыре асинхронных действия без состояния «запрос идёт» — повторное нажатие отправит второй запрос.
S-1 4	Профиль (свой)	ProfileScreen.jsx	Бейджи разделены осмысленно: переписка в «Общении», события в «Профиле».

ID	Экран	Файл	Результат разбора
S-1 5	Профиль (чужой)	UserProfileView.jsx	Дневник друга приходит через visible_diary — состав полей задан на сервере, лишнего клиент не получает.
S-1 6	Публичный профиль	PublicProfile.jsx	Счётчик записей считает только видимые спрашивающему — закрытый профиль не обещает того, чего не покажет.
S-1 7	Мои данные профиля	MyProfileSheet.jsx	Смена ника идёт через claim_username с проверкой занятости в базе.
S-1 8	Настройки	SettingsScreen.jsx	Корень настроек, логики почти нет.
S-1 9	Панели настроек	SettingsPanels.jsx	Выгрузка данных не включает переписку, мысли и социальный граф. Удаление аккаунта не трогает хранилище. → PRIV-003, PRIV-002
S-2 0	Приватность	social/PrivacyHub.jsx	Каждый тумблер действительно пишет настройку на сервер, и её же читает RLS — это проверено. Но оптимистичный откат может разойтись с сервером при ошибке. → UX-002
S-2 1	Списки связей	social/RelationListScreen.js x	Списки приходят через list_relation; чужие списки недоступны.
S-2 2	Просьбы о подписке	social/FollowRequestsScre en.jsx	Просьбы видны только участникам пары — политика follow_requests это обеспечивает.
S-2 3	События	NotificationsScreen.jsx	Уведомления нельзя подделать: политики INSERT нет, push_notification отозвана у authenticated.
S-2 4	Поиск людей	UserSearch.jsx	Поиск по префиксу, минимум два символа, не более 30 записей, заблокированные исключены. Ограничения частоты нет. → UX-003
S-2 5	Челленджи	ChallengesScreen.jsx	Участие проверяется функцией in_challenge во всех трёх политиках таблиц челленджей.
S-2 6	Тренер	CoachScreen.jsx	Доступ тренера к дневнику выдаётся поимённо и не зависит от закрытости аккаунта — осознанное и верное решение.
S-2 7	Добавки	SupplementSheet.jsx	Состав добавки уезжает в запись снимком, уже умноженным на дозу: уточнение справочника не меняет прошлые дни. Сделано верно.
S-2 8	Сканер штрихкода	BarcodeScanner.jsx	Обработаны все шесть ветвей отказа, в каждой есть выход «Добавить вручную». Образцовая работа с отказами.
S-2 9	Вход и регистрация	AuthSheet.jsx	prompt=select_account у Google — иначе «войти другим аккаунтом» невозможно. Кнопка Web3 ведёт к выключенной функции. → DEP-001
S-3 0	Сброс пароля	ResetPasswordSheet.jsx	Восстановительная сессия не считается обычным входом: до смены пароля данные аккаунта не грузятся, отмена равна выходу. Сделано верно.
S-3 1	Редактор рецепта	RecipeEditorSheet.jsx	Рецепт — одно блюдо на порции, шаблон — несколько записей. Разделение проведено последовательно.
S-3 2	Правовая информация	LegalSheet.jsx	Показывает пользователям шаблон с красной плашкой и незаполненными полями; три обработчика не упомянуты. → PRIV-001
S-3 3	Карточка «поделиться»	ShareCardSheet.jsx	Картинка собирается на канве и скачивается локально, на сервер не уходит.
S-3 4	Мысли (лента профиля)	ThoughtsFeed.jsx	Изображение мысли остаётся в публичном бакете после удаления записи и аккаунта. → PRIV-002

ID	Экран	Файл	Результат разбора
S-3 5	Экран ошибки	RootErrorBoundary.jsx	Граница ошибок стоит СНАРУЖИ провайдера состояния — падение самого хранилища тоже показывает экран с объяснением, а не белую страницу.

7. Аудит взаимодействий

756 обработчиков разобраны не по одному, а классами: одинаковые по устройству элементы имеют одинаковые дефекты, и перечисление каждой из 405 кнопок отдельной строкой создало бы видимость полноты, а не полноту. Ниже — проверенные классы и результат по каждому.

Проверка	Итог	Что установлено
Действие срабатывает	ОК	Мёртвых кнопок — то есть обработчиков, не делающих ничего, — не обнаружено ни одной.
Повторное нажатие	ЧАСТИЧНО	Большинство форм имеют состояние <code>busy</code> и блокируют кнопку. Без защиты: <code>ConversationInfo</code> (4 действия), <code>PrivacyHub</code> (2), <code>RelationshipSheet</code> (2), <code>SharedMedia</code> (3). Последствия ограничены — все эти операции идемпотентны на сервере, кроме переключений приватности (UX-002).
Двойное создание записи	ОК	Добавление еды создаёт запись с новым идентификатором на клиенте — дубликат возможен, но виден и удаляется. Платёж защищён на стороне Stripe, промокод — первичным ключом, реакция — переключателем на сервере.
Гонка при частых нажатиях	ЧАСТИЧНО	<code>toggleFavorite</code> и <code>saveStackItem</code> принимают решение ДО <code>setState</code> по актуальному состоянию из <code>ref</code> , а обновляющий колбэк остаётся чистым — это правильный приём, и в коде он прокомментирован. Проблема осталась только в <code>PrivacyHub</code> .
Работа без сети	ОК	Дневник полностью работает офлайн, изменения копятся и досылаются. Социальные разделы честно показывают недоступность, а не вечный спиннер.
Ошибка сервера	ОК	Ошибки Postgres приводятся к человеческим текстам через <code>normalizeError</code> и <code>pgErrors</code> ; сырые коды пользователю не показываются.
Без входа	ОК	Разделы, требующие аккаунта, показывают приглашение войти, а не пустой экран.
Истёкшая сессия	ОК	Единственный автоматический выход — по мёртвому токenu обновления; при этом приватные данные немедленно убираются с экрана (обработка <code>AUTH_ERROR</code> в <code>store.jsx</code>).
Возврат назад	ОК	Есть собственный <code>useSwipeBack</code> ; листы закрываются без потери введённого.
Перезагрузка	ОК	Состояние восстанавливается из кэша до ответа сервера — приложение не мигает пустыми экранами.
Отмена и повторное открытие	ОК	Листы монтируются по условию, то есть состояние не переживает закрытие. Устаревшего состояния при повторном открытии не будет.
Пустой и неверный ввод	ОК	<code>sanitizeAmount</code> снимает буквы, минус и вторую точку; пустые мысли и сообщения отклоняются до отправки.
Очень длинный ввод	ЧАСТИЧНО	Текстовые поля ограничены на сервере (2000 для поддержки, 4000 на реплику AI, 80 на название группы, 300 на память). Числовое количество — не ограничено (UX-001).
Юникод и эмодзи	ОК	<code>normalizeQuery</code> приводит к NFC, снимает диакритику и «ё»; ники приводятся <code>slugify_username</code> ; реакции проверяются по списку допустимых.
Подтверждение удаления	ОК	<code>ConfirmDialog</code> используется в 14 местах — удаление аккаунта, сброс данных, выход из группы, удаление мысли и сообщения. Без подтверждения удаляются только легко восстанавливаемые вещи (приём добавки, свой ответ).
Доступное имя элемента	ЧАСТИЧНО	126 <code>aria-label</code> на 405 кнопок; десять кнопок-символов без имени (A11Y-001).

7.1. Состояния загрузки, пустоты и ошибки

Проверено отдельно, потому что именно здесь чаще всего прячется «вечный спиннер». Результат хороший: в проекте есть осознанное правило — недоступный раздел говорит об этом прямо. Пример из PrivacyHub: «Раздел недоступен. База ещё не обновлена» с кнопкой «Попробовать снова» — вместо бесконечной загрузки. Такой же приём применён в ленте, мыслях и списках.

Запуск приложения намеренно не показывает экран загрузки вовсе: пока неизвестно, есть ли сессия, не рендерится ничего, и интерфейс появляется сразу в нужном виде. Есть две страховки от вечной загрузки — таймер на 6 секунд и гонка `getSession` с таймаутом 4 секунды, чтобы сама страховка не зависла на мёртвой сети.

Не найдено ни одного места, где ошибка проглатывается молча и пользователь остаётся без объяснения. Пустые `catch` есть, но все они относятся к необязательным операциям (вибрация, запись в `localStorage`, отписка от канала) и прокомментированы.

8. Аудит серверных точек

Точка	Вход	Права	Проверка	Лимит	Найдено
POST /api/ai/chat	Bearer	тариф из БД	да	бюджет	P1 SEC-001: проверка остатка и списание не атомарны — параллельные запросы проходят по одному остатку.
POST /api/ai/vision	Bearer	тариф из БД	частично	бюджет	P1 SEC-001 + SEC-007: тип файла проверяется только по заявленному media_type.
POST /api/stripe/checkout	Bearer	свой customer	да	Stripe	Замечаний нет. Адрес возврата сверяется с белым списком (safeOrigin) — прежняя проверка «лишь бы https» закрыта и покрыта тестом.
POST /api/stripe/portal	Bearer	свой customer	да	Stripe	Замечаний нет.
POST /api/stripe/webhook	подпись	Stripe	да	—	P2 SEC-002: неизвестная цена даёт высший тариф. SEC-004: нет контроля порядка событий.
POST /api/telegram/webhook	секрет	список админов	да	—	Замечаний нет. Секрет проверяется первым; отправитель нажатия сверяется со списком отдельно — знание callback_data прав не даёт. Посторонним бот не отвечает вовсе.
POST /api/support	Bearer	свой user	да	1/час в БД	Замечаний нет. Образцовый порядок: origin → вход → бан → частота → запись → отправка. Лимит считается по таблице, а не по памяти процесса.
POST /api/feedback	НЕТ	нет	да	в памяти	P2 SEC-003: открыт без входа, Origin подделывается не-браузером, лимит в памяти и по управляемому клиентом заголовку. CFG-001.

8.1. Общие свойства серверного слоя

- Служебные ключи (ANTHROPIC_API_KEY, SUPABASE_SERVICE_ROLE_KEY, STRIPE_SECRET_KEY, TG_TOKEN) нигде не имеют префикса VITE_ — то есть в клиентский бандл не попадают. Проверено по .env.example и по коду; в .env.local тоже только два клиентских значения плюс ключ Anthropic, а сам файл не отслеживается git и в истории отсутствует.
- Наружу не отдаются ни трассировки, ни сообщения драйвера: все обработчики возвращают обобщённый текст, а подробности пишут в console.error.
- Все точки отклоняют метод, отличный от POST.
- Учёт расхода AI устроен верно по замыслу: резерв до обращения к модели, расчёт по факту после, возврат остатка при ошибке, и отдельный таймаут в 60 секунд именно ради того, чтобы платформа не убила функцию до возврата резерва. Ошибка ровно в одном месте — отброшенном результате (SEC-001).
- Недоступность учёта трактуется как отказ, а не как отсутствие лимита: «отказать дороже, чем раздать бесплатный доступ» — правильный выбор, прокомментированный в коде.

9. Аудит базы данных и RLS

9.1. Сводка

Свойство	Состояние	Комментарий
RLS включён на всех таблицах	33 из 33	Исключений нет.
Таблицы без политик (закрыты полностью)	promo_codes	Намеренно: список действующих кодов не должен быть доступен ни одному клиенту. Верное решение.
SECURITY DEFINER без search_path	0	Все функции с правами определившего защищены. Это лучше типичного состояния проектов на Supabase.
SECURITY INVOKER без search_path	16	Триггеры и ограничители. Прав не повышают — DB-002, гигиена.
Внешние ключи на auth.users	все с каскадом	Удаление пользователя вычищает все 32 связанные таблицы. Проверено сплошным разбором.
Атомарность записи состояния	есть	save_app_state — compare-and-swap по revision.
Ограничения частоты в базе	6 триггеров	Подписки, публикации, ответы, реакции, просьбы, сообщения незнакомым.
Целостность на уровне БД	есть	ЧЕК на диапазоны промокодов, на допустимые значения приватности; UNIQUE на пары; 47 индексов.

9.2. Предикаты доступа — разбор

Три функции определяют почти всю приватность приложения. Все три разобраны построчно; противоречий между ними не обнаружено.

```

can_view_profile_content(owner)          -- «видно ли вообще содержимое»
сам владелец                            → да
блокировка в любую сторону              → НЕТ  (сильнее всего остального)
аккаунт открыт                          → да
иначе                                   → только подписчик

can_view_post(post)                     -- видимость записи
автор                                    → да
блокировка                              → НЕТ
+ can_view_profile_content(автор)      -- закрытость перекрывает круг записи
+ круг: public | followers | friends (взаимная подписка) | close_friends

can_view_diary(owner)                   -- САМОЕ ЧУВСТВИТЕЛЬНОЕ
сам владелец                            → да
блокировка                              → НЕТ
поимённый доступ (diary_access)        → да   } выданы владельцем вручную,
принятый тренер (coach_links)          → да   } закрытость их не отменяет
иначе: can_view_profile_content + круг дневника
(public | followers | mutuals | close_friends)

```

Отдельно отмечу решение, которое легко было сделать неправильно: закрытость аккаунта перекрывает настройку дневника СВЕРХУ. Даже при `diary_visibility = public` закрытый аккаунт не отдаёт дневник посторонним. Это рассуждение записано прямо в комментарии к миграции и реализовано именно так.

9.3. Проверка на подмену личности и IDOR

Целенаправленно проверялись места, где приложение могло бы поверить идентификатору из запроса вместо `auth.uid()`. Ни одного такого места не найдено.

Попытка	Итог	Чем закрыто
Поставить реакцию от чужого имени	НЕТ	<code>toggle_post_reaction</code> и <code>set_message_reaction</code> берут <code>auth.uid()</code> ; <code>user_id</code> в запросе не принимается вовсе.
Отправить сообщение от чужого имени	НЕТ	Политика <code>insert: auth.uid() = sender</code> .
Изменить чужое сообщение	НЕТ	Получателю разрешён только <code>read_at</code> , и то без обнуления — <code>guard_message_update</code> перечисляет все запрещённые поля поимённо.
Удалить чужую запись или ответ	НЕТ	Политики <code>delete</code> проверяют авторство; автор записи вправе удалять ответы под своей записью — это осознанно.
Прочитать чужую переписку по <code>id</code> диалога	НЕТ	<code>is_conversation_member</code> во всех политиках и RPC.
Подделать уведомление	НЕТ	Политики <code>INSERT</code> нет; <code>push_notification</code> отозвана у <code>authenticated</code> .
Узнать, кто поставил реакцию	НЕТ	<code>post_reactions select</code> — только свои строки; счётчики приходят агрегатом из RPC.
Прочитать чужой профиль напрямую	НЕТ	<code>profiles select</code> — только свой; чужие только через <code>user_profile</code> с проверками.
Затащить в группу того, кто заблокировал	НЕТ	<code>can_invite_to_group</code> проверяет блокировку, <code>create_group_conversation</code> — повторно.
Написать в группу заблокировавшему	ДА	SEC-005 — единственная найденная брешь в модели блокировки.
Повысить себе тариф из клиента	НЕТ	<code>tierOf</code> читает базу; вебхук требует подписи Stripe.
Погасить промокод дважды	НЕТ	Первичный ключ (<code>user_id</code> , <code>code</code>) плюс <code>FOR UPDATE</code> .
Превысить <code>max_uses</code> промокода гонкой	НЕТ	<code>FOR UPDATE</code> + <code>CHECK</code> в таблице. Два независимых рубежа.
Потратить чужой бюджет AI	НЕТ	Расход пишется по <code>user.id</code> из проверенного токена.
Превысить свой бюджет AI	ДА	SEC-001 — параллельными запросами.

9.4. Хранилище файлов

Бакет	Публичный	Чтение	Оценка
dm-media	нет	<code>is_conversation_member</code> по первому сегменту пути	Сделано правильно: закрытый бакет, подписанная ссылка на час, запись только в свою папку внутри своего диалога. Образец для остальных.
post-images	ДА	без предиката — доступен всем, включая невошедших	P1 PRIV-002: ссылка-пропуск бессрочна и не отзывается; объект не удаляется ни при удалении записи, ни при удалении аккаунта.
chat-images	ДА	без предиката	P1 PRIV-002: то же. Бакет считается старым (новые вложения идут в dm-media), но продолжает использоваться для фото в чате.

Справедливости ради: ограничения на размер (3 МБ) и типы файлов заданы на самом бакете, а не только в клиенте — прежняя ошибка «ограничение живёт в интерфейсе, то есть его нет» была найдена и исправлена автором самостоятельно. Путь файла содержит `uuid v4`, то есть перебором ссылка не находится.

10. Модель приватности — ожидаемое против фактического

Ключевая проверка этого раздела: то, что скрыто в интерфейсе, должно быть недоступно и через API. Скрытие на клиенте границей доступа не является. Ниже сверка ожидаемой модели с реализацией.

Данные	Чужой	Подписчик	Друг	Близкий	Заблок.	Владелец	Совпадает
Ник, имя, аватар	●	●	●	●	—	●	да
Счётчики подписок	●	●	●	●	—	●	да
Био, «guilty pleasure»	○	●	●	●	—	●	да
Мысли public	○	●	●	●	—	●	да
Мысли followers	—	●	●	●	—	●	да
Мысли friends	—	—	●	●	—	●	да
Мысли close_friends	—	—	—	●	—	●	да
Изображения мыслей	●!	●	●	●	●!	●	НЕТ — PRIV-002
Дневник питания	—	○	○	○	—	●	да
Личные сообщения	—	—	—	—	—	●	да
Сообщения в группе	—	—	—	—	●!	●	НЕТ — SEC-005
Присутствие «в сети»	○	○	○	○	—	●	да
Цель по калориям	—	○	○	○	—	●	да
Список близких друзей	—	—	—	—	—	●	да
Кто поставил реакцию	—	—	—	—	—	○	да

● — доступно · ○ — зависит от настройки владельца · — недоступно · ●! — доступно ВОПРЕКИ ожидаемой модели. «Чужой» — любой вошедший, не связанный с владельцем.

Вывод по разделу

Из пятнадцати проверенных видов данных модель расходится с реализацией в двух местах, и оба связаны не с логикой прав, а с местами, где эта логика не применяется: файлы в хранилище и сообщения в группах. Сама модель прав выстроена верно.

11. Инварианты системы

Восемнадцать утверждений, которые приложение обязано соблюдать всегда. Каждое проверялось попыткой сломать его по коду.

№	Утверждение	Чем обеспечено	Итог
И-0 1	Дневник виден только тем, кого впустил владелец	can_view_diary + политика app_state	держится
И-0 2	Заблокированный не может написать в личные сообщения	can_message + is_blocked_between	держится
И-0 3	Заблокированный не виден заблокировавшему нигде	политика messages select	нарушается в группах (SEC-005)
И-0 4	Реакция принадлежит вошедшему, а не user_id из запроса	toggle_post_reaction: auth.uid()	держится
И-0 5	Сообщение нельзя отправить от чужого имени	политика insert: auth.uid() = sender	держится
И-0 6	Получатель может менять только read_at	guard_message_update	держится
И-0 7	Уведомление нельзя подделать	нет политики INSERT; push_notification revoked от authenticated	держится
И-0 8	Промокод нельзя погасить сверх max_uses	for update + CHECK used <= max_uses	держится
И-0 9	Один промокод — одно гашение на человека	первичный ключ (user_id, code)	держится
И-1 0	Клиент не может сам повысить тариф	tierOf читает базу; вебхук проверяет подпись	держится
И-1 1	Тариф не выдаётся при неизвестной цене	TIER_BY_PRICE[priceld] FREE	нарушается (SEC-002)
И-1 2	FREE не тратит больше дневного бюджета	checkBudget + reserve	нарушается при параллелизме (SEC-001)
И-1 3	Состояние не перезаписывается устаревшей версией	save_app_state: where revision = base	держится
И-1 4	Удалённая запись не воскресает при слиянии	тумбстоуны в syncModel	держится
И-1 5	Данные ушедшего не остаются на устройстве	clearCache по владельцу при выходе	держится (кроме недосланного)
И-1 6	Удаление аккаунта стирает все его данные	каскады от auth.users	нарушается для хранилища (PRIV-002)
И-1 7	Закрытый аккаунт не отдаёт содержимое посторонним	can_view_profile_content	держится
И-1 8	Приватный ответ не попадает в кэш service worker	sw.js: чужой origin и /api/ не трогаются	держится

Пятнадцать из восемнадцати держатся. Три нарушенных — SEC-005, SEC-002 и SEC-001 плюс PRIV-002 — все разобраны в разделе находок.

12. Офлайн, многоустройственность и конфликты

Это сильнейшая часть проекта, и разбирать её стоит не в поисках дефектов, а чтобы зафиксировать, что именно сделано правильно — чтобы при будущих правках это не сломали.

12.1. Механика

- Гибридные логические часы (lib/hlc.js) — один экземпляр на всё приложение, метки монотонно растут независимо от того, кто вошёл и сколько раз пересобралось дерево React.
- Метка времени берётся ДО setState — иначе строгий режим React, вызывающий обновляющую функцию дважды, сжигал бы две метки на одну правку. Это прокомментировано в коде.
- Каждое удаление оставляет тумбстоун. Без него запись воскресала бы из копии другого устройства при следующем слиянии — и в проекте есть тесты ровно на это («удалённый шаблон не воскресает со второго устройства»).
- Скаляры дня версионизируются каждый своей меткой (dayFieldTs): правка настройки на телефоне не конфликтует с добавлением еды на компьютере.
- Кэш разделён по пользователю: ключ eataps:state:<uid>. Перепутать аккаунты физически нечем — прежняя схема с одним ключом и меткой «чьи это данные» была источником показа чужого дневника и заменена.
- Гостевые данные «усыновляются» при первом входе ровно один раз, после чего гостевой кэш стирается.
- Запись на сервер — compare-and-swap: update ... where revision = p_base_revision. Проигравший получает состояние сервера и сливает его локально.
- Соседние вкладки синхронизируются через событие storage и собственную шину tabBus: вход и выход в одной вкладке доходят до остальных, чего supabase-js сам не делает.
- Возврат на вкладку и восстановление сети запускают контрольную сверку: realtime сам по себе не гарантирует, пока вкладка спала.
- Новая версия PWA применяется только когда всё сохранено: setUpdateSafetyCheck не даёт перезагрузить страницу посреди отправки.
- Уход со страницы (pagehide) пытается дослать накопленное; не успели — изменения остаются в кэше и уедут при следующем запуске.

12.2. Разбор сценария

Устройство А (офлайн)	Устройство В (онлайн)
T1 правит завтрак	—
T2 —	правит ужин, revision 5 → 6
T3 А выходит в сеть	
T4 А отправляет с base=5 → конфликт: на сервере уже 6	
T5 А получает состояние сервера и сливает:	
завтрак — метка А (позже для этого поля)	
ужин — метка В	
T6 А отправляет с base=6 → успех, revision 7	
Итог: обе правки сохранены, потери нет.	

Проверено тестами проекта: «шаблон, созданный на телефоне, доезжает до ноутбука», «снятая на телефоне галочка не возвращается с ноутбука», «правка дозы в стеке побеждает по времени, а не по устройству», «приёмы добавок с двух устройств объединяются, ни один не теряется». Потерянных обновлений, дубликатов и призрачных удалений в этих сценариях не обнаружено.

12.3. Единственная оговорка

При выходе из аккаунта локальный кэш стирается только если все изменения доехали до сервера. Если досылка не удалась (нет сети), кэш **СОХРАНЯЕТСЯ**, а человеку об этом сообщается. Это осознанный выбор в пользу сохранности данных, но на общем устройстве он означает, что дневник предыдущего пользователя остаётся в `localStorage` до его следующего входа. Учитывая, что пользователь предупреждён, это не вынесено в отдельную находку — но стоит знать.

13. PWA, service worker и обновление

13.1. Приватность кэша — проверено отдельно

Это место, где приватные данные утекают чаще всего: кэш переживает выход из аккаунта и достаётся следующему пользователю устройства. Здесь сделано правильно.

fetch-обработчик sw.js — порядок проверок:

1. метод не GET → не трогаем
2. чужой origin (весь Supabase) → не трогаем ← приватные ответы мимо кэша
3. /api/* → не трогаем ← серверные функции мимо кэша
4. ссылка входа / сброса пароля → не трогаем ← одноразовый код не из кэша
5. навигация (HTML) → сеть вперёд, кэш как запасной вариант
6. остальное (хэшированные ресурсы) → кэш вперёд

Ни один приватный ответ в кэш не попадает. Отравление кэша недостижимо: кэшируются только ответы своего origin со статусом 200 и типом basic.

13.2. Обновление версии

Сценарий	Итог	Как обработано
Новая версия при открытой вкладке	OK	Новый service worker НЕ активируется сам: иначе он удалил бы старые чанки, пока вкладка работает на прежней сборке, и ленивый импорт упал бы на 404. Активацией управляет страница.
Перезагрузка посреди отправки	OK	setUpdateSafetyCheck не даёт применить обновление при несохранённых изменениях.
Устаревший index с несуществующим чанком	OK	Навигация идёт network-first, поэтому хэши чанков всегда совпадают с задеплоенными.
Сбой загрузки ленивого чанка	OK	lazyWithReload перезагружает страницу — это правильная реакция на исчезнувший после деплоя чанк.
Старый клиент против новой базы	OK	Есть осознанная обработка: isMissingColumn и isMissingRelation. Пример — публикация мысли при ещё не применённой миграции повторяется без поля visibility, а разделы, которых нет в базе, честно показывают «Раздел недоступен».
Новый клиент против старой базы	OK	Тот же механизм: отсутствующая функция даёт unavailable, а не падение.
Установка приложения	OK	manifest.webmanifest, иконки 192 и 512, apple-touch-icon на месте.
Ресурсы кэшируются поштучно	OK	Не addAll: один отсутствующий значок иначе означал бы, что офлайн-режима нет вовсе.

14. Аудит AI-ассистента

14.1. Поток и разграничение прав

Клиент собирает контекст из СВОЕГО состояния

POST /api/ai/chat, заголовок Authorization: Bearer <токен>

1. getUser(token)

— личность из проверенного токена

2. tierOf(user.id)

— ТАРИФ ИЗ БАЗЫ, не из тела запроса •

3. spentThisPeriod()

— расход за сутки

4. checkBudget()

— решение

└ не атомарно

5. reserve()

— списание верхней оценки

└ SEC-001

6. callClaude()

— ключ только на сервере •

7. settle()

— расчёт по факту, возврат остатка •

Проверка	Итог	Установлено
Вызов без входа	ЗАКРЫТ О	Нет токена — 401 до любых действий.
Трата чужого бюджета	ЗАКРЫТ О	Расход пишется по идентификатору из проверенного токена; клиент его не передаёт.
Подмена тарифа из клиента	ЗАКРЫТ О	tierOf читает subscriptions и promo_grants и берёт лучший из двух — тем же правилом, что и клиент.
Обход лимита FREE	ОТКРЫТ О	SEC-001. Для FREE это 3 запроса в сутки против произвольного числа при параллелизме.
Обход лимита обрывом запроса	ЗАКРЫТ О	Резерв списывается ДО обращения к модели, а собственный таймаут в 60 с гарантирует попадание в catch и возврат остатка. Продумано.
Доступ к данным другого пользователя	ЗАКРЫТ О	Контекст собирает клиент из своего состояния; сервер чужого ничего не подмешивает.
Утечка системного промпта	ВОЗМОЖ НА	Внедрением инструкций, как и в любом подобном приложении. Раскрываются описания тонов и правила тарифов — коммерческой тайны там нет.
Загрузка не-изображения	ЧАСТИЧ НО	SEC-007: проверяется только заявленный тип. Последствия малы — файл нигде не сохраняется.
Огромное изображение	ЗАКРЫТ О	Ограничение в 3,5 МБ на строку base64 посчитано с учётом раздувания и потолка тела функции — то есть человек получает понятный текст, а не платформенную ошибку.
Устаревший контекст	ЗАКРЫТ О	Контекст собирается на момент отправки; глубина истории ограничена тарифом.
Избыточные личные данные	ЗАМЕЧА НИЕ	Уходит профиль целиком, история за несколько дней и фотографии. Обосновано задачей, но не раскрыто в политике — PRIV-001.

14.2. Медицинские границы

Промпт содержит явные границы, и они сформулированы лучше, чем обычно бывает в подобных приложениях: не ставить диагнозы, не назначать и не отменять лекарства; осторожность при беременности, диабете, болезнях ЖКТ, почек и печени, приёме препаратов, возрасте до 16, весе сильно ниже нормы и признаках расстройства пищевого поведения; прямой запрет предлагать дефицит ниже 1200 ккал женщинам и 1500 мужчинам и голодание дольше суток.

Отдельного упоминания заслуживает тон «без цензуры»: он включается только явным согласием (18+) и

содержит инструкцию мгновенно и без мата переходить на спокойный регистр при признаках расстройства пищевого поведения, срыва, самоповреждения или тревоги. Это продуманная работа, а не формальная отписка.

Оговорка

Всё перечисленное — инструкции модели, то есть мягкие границы. Серверной проверки ответа нет, заготовленного ответа на кризисную тему нет, и в правовых текстах нигде не сказано, что ассистент не является медицинской консультацией. Для приложения о питании и весе один рубеж — мало (AI-001).

15. Производительность и качество кода

15.1. Проверено

Проверка	Итог	Установлено
Очистка эффектов	0 из 80	Автоматический разбор всех useEffect: каждый, ставящий слушатель, таймер, наблюдатель или подписку, возвращает функцию очистки. Утечек не найдено.
Утечки realtime	нет	Каналы снимаются при смене пользователя и при выходе (removeAllRealtimeChannels).
Утечки таймеров	нет	Heartbeat присутствия, планировщик напоминаний, таймеры тостов — все с очисткой.
Лишние перерисовки	приемлемо	Значение контекста в useMemo, мутации в useCallback, MessageList обёрнут в мемо.
Дорогие вычисления	ОК	Память о продуктах пересчитывается только при изменении дней, а не на каждый рендер — прокомментировано и обосновано.
Размер сборки	ЗАМЕЧАНИЕ	Главный чанк 1,15 МБ (341 КБ gzip); manualChunks не настроен — PERF-001.
Ленивая загрузка	есть	lazyWithReload и LazyBoundary; тяжёлые экраны и Web3 вынесены в отдельные чанки.
Оптимизация изображений	есть	Сжатие до 1280 px и JPEG q=0.8 перед загрузкой; loading="lazy" на изображениях ленты.

15.2. Замечания к архитектуре кода

Приоритет в этом отчёте — дефекты, а не эстетика, поэтому ниже только то, что имеет практические последствия.

- AddMealSheet.jsx — 2 267 строк, 134 обработчика, 81 кнопка, 46 полей в одном файле. Это не «некрасиво», а практический риск: экран отвечает за поиск, порции, штрихкод, шаблоны, рецепты и ручной ввод сразу, и любая правка в нём затрагивает шесть несвязанных сценариев. Разделение по вкладкам — самый полезный рефакторинг в проекте.
- foods.js — 1 496 строк, где справочник продуктов соседствует с поиском, транслитерацией и обращениями к Open Food Facts. Разделение данных и логики заодно решило бы PERF-001.
- src/lib/exchange.js — 311 строк мёртвого кода (CODE-001).
- Стек Web3 — пять зависимостей ради выключенной функции (DEP-001).
- Дублирование правила «лучший из двух источников доступа»: bestTier в src/lib/subscription.js и то же правило в представлении admin_subscriptions. Это отмечено комментарием в самой миграции — риск известен автору, но остаётся.
- Два представления о допустимом весе: setDayWeight принимает 20–400, онбординг — 30–250. Расхождение безобидно, но при добавлении проверки в computeTargets (BUG-001) стоит свести к одному.

16. Доступность и мобильный интерфейс

Критерий WCAG 2.2 AA	Итог	Установлено
4.1.2 Имя, роль, значение	ЧАСТИЧНО	126 aria-label на 405 кнопок; 10 кнопок-символов без имени — A11Y-001.
1.3.1 Структура и связи	ЧАСТИЧНО	Есть role="radio" и aria-checked в списках выбора (30 использований), но заголовочная структура местами держится на размере шрифта, а не на семантике.
2.5.8 Размер цели	ЧАСТИЧНО	Класс tap44 указывает на осознанную работу с размером, но у двух кнопок явно задан minHeight: 32 — ниже рекомендованных 44.
2.1.1 Доступ с клавиатуры	ЧАСТИЧНО	Управление сделано на <button>, то есть фокусируется штатно. Но onSubmit всего 8 на 105 полей: большинство форм отправляются кнопкой, и ввод с клавиши Enter во многих местах не сработает.
2.3.3 Анимация по запросу	ОК	prefers-reduced-motion учитывается (MessageList и другие).
1.4.3 Контраст	НЕ ПРОВЕРЕНО	Нужны измерения пикселями; в этом аудите не выполнялись.
Безопасные зоны, вырез	ОК	safe-area учитывается; есть собственные useScrollLock и useSheetDrag.
Блокировка прокрутки	ОК	Счётчик открытых листов вместо булева флага — два наложенных листа не разблокируют прокрутку преждевременно.

17. Приватность и соответствие GDPR

17.1. Что делается правильно

- Локальная работа по умолчанию: без аккаунта приложение полностью функционально, и данные не покидают устройство. Это лучшая форма минимизации данных.
- Правовое основание выбрано верно: согласие по ст. 6(1)(а) и ст. 9(2)(а), а не «законный интерес» — для данных о здоровье это единственный корректный вариант.
- Данные о питании прямо названы особой категорией по ст. 9 — многие приложения этого избегают.
- Возрастная граница 16 лет заявлена.
- Трекеры и рекламной аналитики нет вовсе: ни одного стороннего скрипта в index.html.
- Удаление аккаунта работает и вычищает все 33 таблицы каскадами; перед удалением явно стираются мысли и следы в чужих ветках — на случай, если удаление самого аккаунта не пройдёт.
- Разговоры с AI на сервере не хранятся: история приходит с клиента и нигде не оседает. Это осознанное решение против второй копии личных данных.
- Фотографии еды для распознавания не сохраняются в облаке — уходят в теле запроса и живут только в памяти.
- Сессии выходят по умолчанию только с текущего устройства; глобальный выход — отдельное осознанное действие.

17.2. Что не соответствует

Требование	Состояние	Разрыв
Ст. 13 — раскрытие получателей	НЕ ВЫПОЛНЕНО	Anthropic, Stripe, Telegram и Open Food Facts не названы. Anthropic при этом получает данные ст. 9 и фотографии — PRIV-001.
Ст. 30 — реестр обработки	НЕ ВЫПОЛНЕНО	Реестр не знает об AI, оплате и поддержке; упоминает удалённый «публичный ID» — PRIV-001.
Ст. 28 — договоры с обработчиками	НЕ ПОДТВЕРЖДЕНО	Чек-лист из четырёх строк, все четыре отметки пусты; трёх фактических обработчиков в нём нет — PRIV-001.
Ст. 12/13 — ответственное лицо	НЕ ВЫПОЛНЕНО	Вместо имени, адреса, контакта и надзорного органа — плейсхолдеры. Реализовать права субъекта невозможно — PRIV-001.
Ст. 17 — право на удаление	ЧАСТИЧНО	База вычищается полностью, изображения не удаляются никогда — PRIV-002.
Ст. 15/20 — доступ и переносимость	ЧАСТИЧНО	Выгружается одна таблица из 33: нет переписки, мыслей, ответов, реакций, событий и социального графа — PRIV-003.
Ст. 9(2)(а) — отдельное согласие	ЧАСТИЧНО	Согласие на облако заявлено, но отдельного согласия на передачу данных о здоровье в AI нет, хотя это новый получатель и новая цель.
Ст. 5 — минимизация	ХОРОШО	Локальная работа по умолчанию, отсутствие трекеров, отсутствие хранения переписки с AI.

18. Находки

Всего 22: P0 — 0, P1 — 3, P2 — 5, P3 — 12, P4 — 2. Каждая содержит доказательство, воспроизведение, причину, предлагаемое исправление и тест против повторения.

ID	Ур.	Заголовок	Категория	Статус
SEC-001	P1	Дневной лимит AI обходится параллельными запросами: проверка и списание не атомарны	Обход лимита / расход средств	CONFIRMED
PRIV-001	P1	Anthropic, Stripe, Telegram и Open Food Facts не раскрыты ни в политике, ни в реестре обработки; политика опубликована как незаполненный шаблон	GDPR / раскрытие третьих лиц	CONFIRMED
PRIV-002	P1	Загруженные изображения не удаляются никогда — в том числе при удалении поста и всего аккаунта; бакеты открыты на чтение	Право на удаление / хранилище	CONFIRMED
SEC-002	P2	Незаданная переменная цены превращает неизвестный прайс в высший тариф AI_PREMIUM	Подписки / повышение прав	CONFIRMED
SEC-003	P2	/api/feedback открыт без входа, защищён только заголовком Origin и счётчиком в памяти процесса	Злоупотребление / отсутствие аутентификации	CONFIRMED
DB-001	P2	setup_all.sql правлен руками, новая миграция не зарегистрирована в сборке; следующая пересборка молча отменит правку	Целостность развёртывания	CONFIRMED
SEC-004	P2	Вебхук Stripe не защищён от переупорядоченной доставки: позднее старое событие воскрешает отменённый тариф	Подписки / целостность состояния	PROBABLE
PRIV-003	P2	Выгрузка данных не включает переписку, мысли, комментарии, реакции, события и социальный граф	GDPR / переносимость	CONFIRMED
UX-001	P3	Количество продукта не ограничено сверху: 999999999999 г принимаются и портят всю статистику	Целостность данных / ввод	CONFIRMED
BUG-001	P3	computeTargets не проверяет вход: нечисловой профиль даёт NaN во всех целях, Infinity в весе — бесконечные калории	Устойчивость расчёта	CONFIRMED
SEC-005	P3	Блокировка не распространяется на общие групповые диалоги	Приватность / блокировка	CONFIRMED
DEP-001	P3	Неиспользуемый стек Web3 приносит 12 из 17 уязвимостей prnt и чанк на 2,68 МБ в каждой сборке	Зависимости / мёртвый код	CONFIRMED
A11Y-001	P3	Кнопки-символы «←» и «×» не имеют доступного имени	Доступность (WCAG 2.2 AA)	CONFIRMED
SEC-006	P3	CSP разрешает unsafe-inline и unsafe-eval, connect-src открыт на любой https	Защита в глубину	CONFIRMED
SEC-007	P3	Тип загружаемой в AI картинки проверяется только по заявленному media_type, без сигнатуры файла	Проверка входных данных	CONFIRMED
UX-002	P3	Оптимистичные переключатели приватности могут рассинхронизироваться с сервером при быстром переключении и ошибке	Согласованность состояния	PROBABLE
CFG-001	P3	Личные Telegram chat ID записаны в исходном коде и продублированы как значение по умолчанию	Конфигурация / приватность владельца	CONFIRMED

ID	Ур.	Заголовок	Категория	Статус
DB-002	P3	Восемь триггерных функций объявлены без set search_path	Практика написания SQL	CONFIRMED
UX-003	P3	Поиск по префиксу позволяет перебрать базу пользователей; ограничения частоты на поиск нет	Перечисление пользователей	CONFIRMED
AI-001	P3	Границы медицинской безопасности заданы только промптом; серверной проверки ответа нет	Безопасность AI	CONFIRMED
CODE-001	P4	src/lib/exchange.js — 311 строк экспорта и импорта CSV, не подключённых ни к одному экрану	Мёртвый код	CONFIRMED
PERF-001	P4	Главный чанк 1,15 МБ (341 КБ в gzip); разбиение на части не настроено	Производительность	CONFIRMED

SEC-001 — Дневной лимит AI обходится параллельными запросами: проверка и списание не атомарны

P1 **CONFIRMED** · уверенность HIGH · Обход лимита / расход средств

Где `api/ai/_shared.js:88-104 (recordSpend)`, `api/ai/chat.js:37-76`, `api/ai/vision.js:44-82`
 Затронутый поток AI-ассистент → бюджет тарифа

Суть

Порядок «прочитать расход → проверить остаток → зарезервировать» не атомапен. `spentThisPeriod()` читает потраченное, `checkBudget()` решает по прочитанному, и только потом `reserve()` списывает. Между чтением и списанием любое число параллельных запросов проходит проверку по ОДНОМУ И ТОМУ ЖЕ остатку.

Инфраструктура для атомарной проверки уже есть и работает: RPC `ai_usage_add` выполняет `insert ... on conflict do update` и возвращает новый итог (`returning spent_micro into v_total`). Но `recordSpend` деструктурирует только `{ error }` и ВЫБРАСЫВАЕТ `data`. Значит вызывающий код физически не может узнать, что резерв увёл счёт за потолок, и не отменяет запрос.

Доказательство

Проверено по коду:

- `ai_usage_add` возвращает bigint — новый `spent_micro` (`setup_all.sql`, миграция 2026-08-24 `ai_usage`).
- `recordSpend` — `const { error } = await admin().rpc(...)`; `data` не читается, наружу уходит только `true/false`.
- `reserve()` — тонкая обёртка над `recordSpend`, тоже возвращает булево.
- В `chat.js/vision.js` результат резерва проверяется лишь на «записалось или нет», а не «влезли ли в бюджет».

Арифметика для FREE (вычислено запуском `src/lib/aiBudget.js`):
 дневной лимит = $\text{floor}(0.5 \$ \times 1e6 / 30) = 16\ 666$ микродолларов
 один chat-запрос (вход ~1000 токенов, Haiku) = $1000 \times 1 + 700 \times 5 = 4\ 500$
 легальных запросов в день = 3

Воспроизведение

- Войти любым обычным аккаунтом (тариф FREE).
- Взять `access_token` из сессии Supabase.
- Отправить 100 запросов `POST /api/ai/chat` ОДНОВРЕМЕННО (`Promise.all / xargs -P 100`), а не последовательно.

4. Все 100 читают `spent=0`, все проходят `checkBudget`, все резервируют и все уходят в Anthropic.
5. Повторяемо каждые сутки: период — календарный день UTC.

Ожидаемое / фактическое

Ожидается	Не более 3 запросов в сутки на тарифе FREE; запрос сверх лимита получает 402 <code>budget_exhausted</code> ДО обращения к модели.
Фактически	Проходит столько запросов, сколько их запущено параллельно. Перерасход ограничен только степенью параллелизма клиента.

Последствия

Прямой денежный ущерб владельцу ключа Anthropic: платит он, лимит не действует. ~33-кратное превышение при 100 потоках, повторяемо ежедневно и с любого числа бесплатных аккаунтов. Одновременно обесценивается платная подписка: тариф FREE даёт то, за что берут деньги.

Первопричина

Классический TOCTOU. Атомарный примитив написан правильно, но его результат отброшен на стыке слоёв.

Предлагаемое исправление

Вернуть итог из `ai_usage_add` и решать по нему, а не по прочитанному заранее:

```
// _shared.js
export async function recordSpend(userId, micro, period, { count = true } = {}) {
  const delta = Math.round(Number(micro) || 0)
  if (!delta) return { ok: true, total: null }
  const { data, error } = await admin().rpc('ai_usage_add', {...})
  if (error) { console.error(...); return { ok: false, total: null } }
  return { ok: true, total: Number(data) } // ← итог больше не теряется
}

// chat.js / vision.js
const r = await reserve(user.id, reserved, period)
if (!r.ok) return res.status(503).json({ error: 'accounting_unavailable' })
const budget = budgetForTier(tier)
if (budget !== null && r.total > budget) {
  await settle(user.id, { reserved, actual: 0 }, period) // вернуть резерв
  return budgetError(res, { reason: 'exhausted', remaining: 0 }, tier)
}
```

После этого решение принимается по значению, полученному ИЗ атомарной операции, и гонка закрывается без блокировок.

Тест против повторения

Тест на уровне `api/ai`: замочать `ai_usage_add` общим счётчиком, запустить N=50 обработчиков через `Promise.all` при бюджете на 3 запроса, проверить, что `callClaude` вызван не более 3 раз, а остальные получили 402.

PRIV-001 — Anthropic, Stripe, Telegram и Open Food Facts не раскрыты ни в политике, ни в реестре обработки; политика опубликована как незаполненный шаблон

P1 **CONFIRMED** · уверенность HIGH · GDPR / раскрытие третьих лиц

Где src/components/LegalSheet.jsx:38-110,
docs/compliance/verzeichnis-verarbeitungstaetigkeiten.md,
docs/compliance/avv-checkliste.md

Затронутый поток Правовые документы ↔ фактическая обработка данных

Суть

Документы по защите данных описывают приложение, каким оно было до появления AI-ассистента, подписок и поддержки. Раздел «4. Empfänger / Auftragsverarbeiter» перечисляет Vercel, Supabase, Google и Reown/WalletConnect. Фактически данные уходят ещё четырём получателям, и ни один не назван:

- Anthropic PBC — получает профиль (пол, возраст, рост, вес, цели), дневник питания за сегодня, историю за несколько дней, «память» о пользователе и ФОТОГРАФИИ ЕДЫ. Это ровно те особые категории данных по ст. 9 GDPR, которые политика сама и называет особыми в разделе 2.
- Stripe — платёжные данные и email.
- Telegram — обращения в поддержку уходят в чат бота вместе с ником, именем и UUID пользователя.
- Open Food Facts (ru.openfoodfacts.org) — запросы уходят прямо из браузера, то есть третьей стороне видны IP пользователя и что он сканирует.

Отдельно: политика показывается людям с красной плашкой «Шаблон. Проверьте у юриста и дополните своими данными», а вместо ответственного лица, адреса, контакта и надзорного органа стоят [Name], [Anschrift], [E-Mail], [zuständige Landesdatenschutzbehörde], «Stand: [Datum]».

Доказательство

```
grep по docs/compliance/*.md на «Anthropic», «Stripe», «Telegram», «AI», «Claude» — ноль
совпадений во всех пяти документах.
LegalSheet.jsx:68 — список получателей: Vercel, Supabase, Google, Reown.
LegalSheet.jsx:41-43 — плашка «Шаблон» в коде компонента, то есть видна в проде.
src/lib/aiPrompt.js:382-392 (buildUserContext) — состав того, что уходит модели.
api/ai/vision.js:60-66 — фотография уходит в Anthropic в теле запроса.
api/support.js:88-99 — ник, имя, UUID и текст уходят в Telegram.
src/lib/foods.js:1385, 1459 — прямой fetch на ru.openfoodfacts.org из браузера.
avv-checkliste.md — таблица AVV/DPA из 4 строк, все четыре чекбоксы пусты ().
```

Воспроизведение

1. Открыть приложение → Профиль → Правовая информация → Datenschutz.
2. Прочитать раздел 4 «Empfänger / Auftragsverarbeiter».
3. Сверить со списком реальных исходящих запросов в DevTools → Network при использовании AI-ассистента и сканера штрихкодов.

Ожидаемое / фактическое

Ожидается Все обработчики персональных данных перечислены в политике и в реестре по ст. 30; с каждым заключён договор по ст. 28; для передач в третьи страны указано основание.

Фактически Четыре обработчика не названы нигде, включая того, кто получает данные о здоровье и фотографии. Реестр по ст. 30 и чек-лист AVV не знают об AI, оплате и поддержке. Политика опубликована в виде шаблона с плейсхолдерами.

Последствия

Для приложения с немецкими правовыми текстами (то есть нацеленного на рынок ЕС), обрабатывающего данные по ст. 9, это нарушение ст. 13, 30 и, при отсутствии договоров, ст. 28 GDPR. Отсутствие названного

ответственного лица и контакта делает невозможной реализацию прав субъекта — то самое, что проверяется в первую очередь по жалобе. Это блокирующий пункт для публичного запуска в ЕС, а не косметика.

Первопричина

Документы написаны один раз на раннем этапе и не обновлялись при добавлении AI, Stripe и Telegram. Ни одной автоматической связи между «появился новый исходящий запрос» и «дополнить реестр» нет.

Предлагаемое исправление

До запуска:

1. Дополнить LegalSheet → Datenschutz раздел 4: Anthropic PBC (США, SCC) — обработка данных о питании и фотографий для функции ассистента; Stripe Inc. (США) — платежи; Telegram (обращения в поддержку); Open Food Facts (запрос сведений о продукте по штрихкоду).
2. В разделе 2 добавить фотографии еды как отдельную категорию.
3. Дополнить verzeichnis-verarbeitungstaetigkeiten.md двумя новыми видами деятельности: «AI-ассистент» (ст. 9(2) (a), получатель Anthropic) и «Поддержка и модерация» (получатель Telegram), плюс «Оплата подписки» (Stripe).
4. Внести Anthropic, Stripe, Telegram в avv-checkliste.md и заключить DPA (у Anthropic и Stripe они стандартные, принимаются в кабинете).
5. Заполнить все [плейсхолдеры] и снять красную плашку «Шаблон» — пока она есть, документ юридически не является политикой.
6. Убедиться, что согласие на AI берётся отдельно: это передача данных ст. 9 новому получателю, и молчаливым оно быть не может.

Тест против повторения

Тест-сторож: список внешних хостов, к которым обращается код (fetch/axios/SDK), собирается grep-ом и сверяется со списком получателей в LegalSheet.jsx и в реестре. Новый хост без записи в обоих документах — падение теста. По образцу существующего sqlGuard.test.js.

PRIV-002 — Загруженные изображения не удаляются никогда — в том числе при удалении поста и всего аккаунта; бакеты открыты на чтение

P1 **CONFIRMED** · уверенность HIGH · Право на удаление / хранилище

Где src/lib/supabase.js:331-337 (deletePost), :322-330 (updatePost), :428-446 (uploadImage), src/lib/supabase.js deleteAccount, supabase/setup_all.sql:473-500, 2100-2128

Затронутый поток Мысли с фото · фото в чате · удаление аккаунта

Суть

Ни один путь в приложении не удаляет объект из хранилища. `deletePost` стирает строку в таблице posts, `updatePost` заменяет ссылку на новую, `unsend\_message` обнуляет image_url, `deleteAccount` удаляет посты, комментарии, реакции, app_state и самого пользователя — но сам файл в бакете во всех четырёх случаях остаётся.

Бакеты `post-images` и `chat-images` созданы с `public = true`, а политика чтения — `for select using (bucket\_id = 'post-images')`, то есть без единого условия: файл доступен любому, включая неаутентифицированного, по прямой ссылке.

Автор осознанно выбрал модель «ссылка-пропуск» (путь содержит uuid v4, ~122 бита, не перебирается) и честно записал это в комментарии к миграции. Претензия не к выбору модели, а к тому, что у пропуска нет срока и его нельзя отозвать: ссылка, один раз попавшая к подписчику, переживает и отписку, и блокировку, и удаление поста, и удаление аккаунта.

Доказательство

```
supabase/setup_all.sql:479-481 — `create policy "chat-images read" ... for select using (bucket_id = 'chat-images');` — предикат отсутствует.
supabase/setup_all.sql:2110-2111 — то же для post-images.
supabase/setup_all.sql:474, 2100 — `insert into storage.buckets (id, name, public) values (... , true)` .
src/lib/supabase.js:331 — тело deletePost целиком: один `.from('posts').delete().eq('id', postId)` , обращения к storage нет.
grep по src/ на `storage.from(...).remove(` — единственное совпадение отсутствует; `.remove(` встречается только как `classList.remove`.
deleteAccount — удаляет 6 таблиц, storage не трогает.
Для сравнения: бакет `dm-media` (setup_all.sql:12460) сделан правильно — `public = false`, чтение через `is_conversation_member`, клиент получает подписанную ссылку на час (src/lib/messaging.js:327).
```

Воспроизведение

1. Пользователь А публикует мысль с фотографией, видимость «только подписчикам».
2. Подписчик В открывает ленту, из DevTools копирует URL изображения.
3. А блокирует В, затем удаляет мысль, затем удаляет аккаунт целиком.
4. В (или кто угодно, кому В передал ссылку, в том числе без входа в приложение) открывает тот же URL — изображение отдаётся.

Ожидаемое / фактическое

Ожидается	Удаление поста, отмена сообщения и удаление аккаунта делают изображение недоступным.
Фактически	Изображение остаётся публично доступным бессрочно. Никакого способа его удалить у пользователя нет.

Последствия

Нарушение права на удаление (ст. 17 GDPR): после удаления аккаунта персональные данные — фотографии, в том числе с людьми и обстановкой, — продолжают отдаваться с сервера проекта. Дополнительно: фото из «закрытых» мыслей не защищены ничем, кроме неразглашения ссылки, и бесконечный рост платного хранилища.

Первопричина

Жизненный цикл файла не связан с жизненным циклом строки, которая на него ссылается. Бакет `dm-media` был позже сделан закрытым, но два ранних бакета остались публичными и без уборки.

Предлагаемое исправление

Короткий путь (снимает утечку после удаления):

1. Хранить в posts не публичный URL, а путь в бакете (`user\_id/uuid.jpg`).
2. Удалять объект вместе со строкой. Надёжнее — триггером в базе через очередь на удаление, а не из клиента: клиент может закрыть вкладку между двумя вызовами.
3. В deleteAccount перед удалением аккаунта выгрузить список объектов пользователя (`storage.from(b).list(uid)`) и удалить их.

Правильный путь (снимает и бессрочный пропуск):

4. Перевести post-images и chat-images на модель dm-media: `public = false`, политика чтения через `can\_view\_post` / участие в диалоге, клиент получает подписанную ссылку. Тогда отписка и блокировка начинают действовать и на картинки.
- Миграция существующих файлов: бакет переключается в private одной строкой, ссылки в старых постах переписываются на пути пакетным скриптом.

Тест против повторения

Интеграционный тест: создать пост с изображением → удалить пост → запросить URL изображения → ожидать 404/403. Аналогично для deleteAccount. Плюс сторож: политика чтения бакета без предиката — падение теста policy-audit.mjs.

SEC-002 — Незаданная переменная цены превращает неизвестный прайс в высший тариф AI_PREMIUM

P2 **CONFIRMED** · уверенность HIGH · Подписки / повышение прав

Где api/stripe/_shared.js:26-35 (TIER_BY_PRICE), api/stripe/webhook.js:46-48 (upsertSubscription)

Затронутый поток Вебхук Stripe → тариф пользователя

Суть

Карта прайс-ID → тариф строится вычисляемыми ключами из переменных окружения. Если переменная не задана, ключом становится СТРОКА "undefined". В вебхуке тариф ищется как `TIER\_BY\_PRICE[priceId]`; когда priceId равен undefined, JavaScript приводит ключ к той же строке "undefined" — и находит запись.

Последняя запись в объекте — AI_PREMIUM. То есть при незаданной STRIPE_PRICE_AI_PREMIUM отказ определить цену подписки даёт не FREE, а САМЫЙ ВЫСОКИЙ тариф: безлимитный бюджет и модель Sonnet вместо Haiku. Отказоустойчивость направлена в сторону, противоположную безопасной.

STRIPE_PRICE_AI_PREMIUM не упомянута в .env.example (там задокументированы только STRIPE_PRICE_AI и STRIPE_PRICE_AI_PLUS), то есть в проде она с высокой вероятностью не задана.

Доказательство

Проверено запуском на реальной семантике JS:

```
process.env.STRIPE_PRICE_AI = 'price_ai_123'
process.env.STRIPE_PRICE_AI_PLUS = 'price_plus_456'
delete process.env.STRIPE_PRICE_AI_PREMIUM
TIER_BY_PRICE = { [env.AI]: 'AI', [env.PLUS]: 'AI_PLUS', [env.PREMIUM]: 'AI_PREMIUM' }
```

```
ключи карты: ["price_ai_123", "price_plus_456", "undefined"]
TIER_BY_PRICE[undefined] → "AI_PREMIUM"
mappedTier = TIER_BY_PRICE[undefined] || 'FREE' → "AI_PREMIUM"
```

.env.example: STRIPE_PRICE_AI и STRIPE_PRICE_AI_PLUS есть, STRIPE_PRICE_AI_PREMIUM отсутствует.
webhook.js:46 — `const priceId = subscription?.items?.data?.[0]?.price?.id` — цепочка опциональна, то есть undefined здесь штатно возможен.

Воспроизведение

Требуется событие подписки, у которого не читается items.data[0].price.id (подписка без элементов, отличие формы объекта между версиями API, частично развёрнутый объект в полезной нагрузке вебхука). Статус при этом должен быть живым (active/trialing/past_due). Прямой проверки на боевом Stripe не проводилось — оценка сделана по коду, поэтому эксплуатируемость помечена как требующая подтверждения, а сам дефект подтверждён.

Ожидаемое / фактическое

Ожидается Не удалось определить тариф по цене → FREE.

Фактически

Не удалось определить тариф по цене → AI_PREMIUM, безлимит.

Последствия

Бесплатная выдача высшего тарифа. В деньгах это безлимитный расход на Sonnet за счёт владельца. Эксплуатируемость зависит от формы события Stripe и оценивается как низкая, но цена ошибки высока, а исправление — одна строка.

Первопричина

Вычисляемые ключи объекта молча принимают undefined и приводят его к строке. Отсутствует явная проверка, что priceId вообще есть.

Предлагаемое исправление

Строить карту только из заданных переменных и требовать явного priceId:

```
export const TIER_BY_PRICE = Object.fromEntries([
  [process.env.STRIPE_PRICE_AI, 'AI'],
  [process.env.STRIPE_PRICE_AI_PLUS, 'AI_PLUS'],
  [process.env.STRIPE_PRICE_AI_PREMIUM, 'AI_PREMIUM']]
  .filter(([id]) => typeof id === 'string' && id.length > 0)
)

// webhook.js
const priceId = subscription?.items?.data?.[0]?.price?.id
if (!priceId) console.error('[stripe/webhook] у подписки нет цены', subscription?.id)
const mappedTier = (priceId && TIER_BY_PRICE[priceId]) || 'FREE'

И дополнительно задокументировать STRIPE_PRICE_AI_PREMIUM в .env.example.
```

Тест против повторения

Юнит-тест рядом с origin.test.js: при незаданной STRIPE_PRICE_AI_PREMIUM карта не содержит ключа "undefined", а upsertSubscription с подпиской без price даёт tier === 'FREE'.

SEC-003 — /api/feedback открыт без входа, защищён только заголовком Origin и счётчиком в памяти процесса

P2 **CONFIRMED** · уверенность HIGH · Злоупотребление / отсутствие аутентификации

Где

api/feedback.js:1-110, api/stripe/origin.js:96-107 (isAllowedOrigin)

Затронутый поток

Кнопка «Совет по приложению» → Telegram владельца

Суть

Точка не требует входа. Единственный заслон — заголовок Origin, сверяемый со списком. Это работает против чужой ВЕБ-СТРАНИЦЫ (браузер не даёт подделать Origin), но ничего не значит для запроса не из браузера: curl ставит любой заголовок.

Ограничение частоты живёт в `const hits = new Map()` — в памяти экземпляра функции. На бессерверной платформе экземпляров много, они создаются под нагрузкой и умирают; общий счётчик отсутствует. Ключ — первый элемент X-Forwarded-For, то есть значение из заголовка запроса.

Автор честно описал слабость лимита в комментарии, но не описал, что сам заслон по Origin от не-браузера не спасает, — и в сумме точка остаётся открытой.

Доказательство

```
api/feedback.js:88 — проверка `isAllowedOrigin(origin)`, никакой проверки токена рядом нет
(сравните с api/support.js:32, где стоит `getUserFromRequest`).
isAllowedOrigin — чистое сравнение строки со списком, никакой криптографии.
api/feedback.js:99 — `const ip = String(req.headers['x-forwarded-for'] ||
 '').split(',')[0].trim()` — берётся ЛЕВЫЙ элемент, то есть та часть цепочки, которую клиент может
задать сам.
api/feedback.js:14 — `const hits = new Map()` на уровне модуля.
api/feedback.js:5 — `const CHAT_IDS = [571138125, 938539456]` — адресаты заданы в коде.
```

Воспроизведение

```
curl -s -X POST https://www.eataps.com/api/feedback \
-H 'Content-Type: application/json' \
-H 'Origin: https://www.eataps.com' \
-H "X-Forwarded-For: 10.0.0.$((RANDOM%255))" \
-d '{"text":"проверка"}'
```

Запустить в цикле с параллелизмом: каждый запрос со своим X-Forwarded-For попадает в отдельную корзину, а холодные экземпляры начинают счёт заново. Проверка проводилась ТОЛЬКО чтением кода; по боевому адресу запросы не отправлялись.

Ожидаемое / фактическое

Ожидается	Отправить совет можно из приложения с разумной частотой; массовая рассылка в чат владельца невозможна.
Фактически	Любой, кто знает адрес, отправляет сообщения в личный Telegram владельца практически без ограничений.

Последствия

Затопление личного чата владельца (и второго адресата) — вплоть до того, что настоящие обращения из поддержки в этом потоке теряются. Расход квоты Telegram Bot API и выполнения функций. Прямой утечки данных нет.

Первопричина

Заголовок Origin принят за границу доступа, а не за один из слоёв; ограничение частоты сделано на структуре, не переживающей архитектуру платформы.

Предлагаемое исправление

Кратко: либо потребовать вход, как в api/support.js, либо перенести лимит в общее хранилище.

- Самое дешёвое и правильное: `const user = await getUserFromRequest(req); if (!user) return res.status(401)...` — «совет по приложению» вполне может требовать аккаунта, как и поддержка.
- Если анонимность нужна принципиально — вести лимит в таблице Postgres (как уже сделано в api/support.js через support_messages) или включить Vercel Firewall / BotID для этого маршрута.
- Ключ лимита брать не из левого элемента XFF, а из значения, которому можно верить на этой платформе.
- Вынести CHAT_IDS в переменную окружения — TG_ADMIN_CHAT_IDS уже существует ради этого в api/telegram/_tg.js.

Тест против повторения

Тест: 10 последовательных вызовов обработчика с разными X-Forwarded-For при одном и том же аккаунте/адресе должны дать 429 начиная с четвёртого — сегодня дают 200.

DB-001 — setup_all.sql правлен руками, новая миграция не зарегистрирована в сборке; следующая пересборка молча отменит правку

P2 **CONFIRMED** · уверенность HIGH · Целостность развёртывания

Где supabase/setup_all.sql:5740, 7410;
supabase/migrations/2026-09-11_fav_restaurant.sql; scripts/build-setup-all.mjs:30-54 (SOURCES)

Затронутый поток Установка базы с нуля · обновление прода

Суть

Файл полной установки собирается скриптом из списка SOURCES и снабжён шапкой «ФАЙЛ СОБИРАЕТСЯ АВТОМАТИЧЕСКИ. Не правьте его руками». Тем не менее в рабочем дереве он правлен руками: в ДБЕ исторические копии функции visible_diary дописана строка ``favRestaurant`, a.state->'profile'->'favRestaurant',`.

Новая миграция 2026-09-11_fav_restaurant.sql, которая делает то же самое штатным способом, в SOURCES НЕ добавлена. Получилось худшее из двух состояний: собранный файл разошёлся с источниками, а источник, который должен был это закрыть, в сборке не участвует.

Собственный тест проекта это ловит и падает — но только он один из 814.

Доказательство

```
npm test → 813 пройдено, 1 провален:
× setup_all.sql собран из источников и не разошёлся с ними (src/lib/sqlGuard.test.js:37)
AssertionError: supabase/setup_all.sql разошёлся с источниками.
npm run sql:verify → падает на первом же шаге.
git diff supabase/setup_all.sql → 2 добавленные строки на 5740 и 7410.
grep -c fav_restaurant scripts/build-setup-all.mjs → 0.
При этом сама миграция корректна и статическую проверку проходит: node scripts/check-sql.mjs → ●
для 2026-09-11_fav_restaurant.sql.
```

Воспроизведение

1. `node scripts/build-setup-all.mjs` (обычный шаг при любом изменении SQL).
2. Скрипт пересоберёт setup_all.sql строго из SOURCES.
3. Обе ручные правки исчезнут, миграции с favRestaurant в списке нет.
4. База, поднятая с нуля по новому setup_all.sql, не отдаёт favRestaurant в visible_diary — поле профиля молча пропадает у друзей, хотя в интерфейсе оно есть.

Ожидаемое / фактическое

Ожидается setup_all.sql всегда равен сборке из SOURCES; новая миграция входит в SOURCES.

Фактически Файл расходится с источниками, миграция вне сборки, sql:verify и npm test красные.

Последствия

Расхождение схемы между новой установкой и прод-базой — тот самый класс ошибок, ради которого в проекте написаны build-setup-all.mjs и sqlGuard.test.js. Правка теряется незаметно: тест упадёт, но конечная схема окажется другой. Плюс на ветке лежит красная сборка.

Первопричина

Незавершённая работа: миграция написана, в список не внесена, а результат вписан в собранный файл вручную — ровно тот путь, который шапка файла запрещает.

Предлагаемое исправление

1. Отменить ручные правки: ``git checkout -- supabase/setup_all.sql``.
2. Добавить `'supabase/migrations/2026-09-11_fav_restaurant.sql'` последней строкой в `SOURCES` (`scripts/build-setup-all.mjs`).
3. ``node scripts/build-setup-all.mjs``.
4. ``npm test`` — `sqlGuard` должен позеленеть.
5. Не коммитить, пока ``npm run sql:verify`` не проходит.

Тест против повторения

Уже существует и работает: `src/lib/sqlGuard.test.js`. Достаточно не допускать коммита с красным тестом (git hook или проверка в CI).

SEC-004 — Вебхук Stripe не защищён от переупорядоченной доставки: позднее старое событие воскрешает отменённый тариф

P2 **PROBABLE** · уверенность MEDIUM · Подписки / целостность состояния

Где `api/stripe/webhook.js:88-130`, `upsertSubscription:43-62`
Затронутый поток Жизненный цикл подписки

Суть

Каждое событие подписки делает безусловный `upsert` строки целиком. Ни ``event.created``, ни ``updated_at`` строки в базе не сравниваются с тем, что уже записано. Stripe явно не гарантирует порядок доставки и повторяет неподтверждённые события.

Значит последовательность «`updated (active)` → `deleted (canceled)`», доставленная в обратном порядке или с повтором первого после второго, оставит в базе `tier` активного тарифа.

Доказательство

`webhook.js:43-62` — `upsertSubscription` не читает текущую строку и не сравнивает временные метки. Защита от повтора держится только на 5-минутном допуске подписи в `constructEvent` — он отсекает старый повтор, но не меняет порядок доставки внутри окна. Проверка проводилась чтением кода; воспроизведение на боевом Stripe не выполнялось — отсюда статус **PROBABLE**.

Воспроизведение

Смоделировать двумя вызовами обработчика с корректной подписью: сначала событие `customer.subscription.deleted (status canceled)`, затем `customer.subscription.updated` со СТАРЫМ `event.created` и `status active`. В базе останется живой тариф.

Ожидаемое / фактическое

Ожидается Событие, более старое, чем уже применённое к этой подписке, игнорируется.
Фактически Применяется последнее пришедшее, независимо от времени его создания.

Последствия

Продолжение платного доступа после отмены или неоплаты. Денежные потери ограничены, но восстановление состояния ручное. Обратный случай (отмена «затирает» продление) бьёт по платящему пользователю.

Первопричина

Отсутствует контроль версии/времени при записи — при том, что ровно такой контроль правильно сделан в соседней части проекта (save_app_state с compare-and-swap по revision).

Предлагаемое исправление

Добавить колонку и сравнение:

```
alter table public.subscriptions
add column if not exists last_event_at timestamptz;
```

```
// webhook.js
await admin().from('subscriptions').upsert({ ..., last_event_at: new Date(event.created *
1000).toISOString() })
// и отбрасывать событие, если в строке уже стоит более поздний last_event_at
```

Надёжнее — перенести запись в RPC с условием `where last\_event\_at is null or last\_event\_at <= \$new`, чтобы проверка и запись были одной атомарной операцией, как это сделано в save_app_state.

Тест против повторения

Тест обработчика: применить событие с created = T2, затем с created = T1 (T1 < T2); строка обязана сохранить состояние из T2.

PRIV-003 — Выгрузка данных не включает переписку, мысли, комментарии, реакции, события и социальный граф

P2 **CONFIRMED** · уверенность HIGH · GDPR / переносимость

Где src/components/SettingsPanels.jsx:526-548 (exportData)

Затронутый поток Настройки → Данные → Выгрузка

Суть

Кнопка выгрузки формирует JSON из состояния приложения: профиль, тема, дни, свои продукты и ингредиенты, недавние, настройки, подписка. Всё, что живёт в отдельных таблицах, в файл не попадает: сообщения, мысли, ответы, реакции, уведомления, подписки/подписчики, списки близких друзей и доступа к дневнику, обращения в поддержку.

К чести реализации, это не скрыто: под кнопкой прямо написано «Переписка и мысли в него не входят — они хранятся отдельно». Но честная оговорка не заменяет права: ст. 15 и 20 GDPR говорят обо ВСЕХ персональных данных, а не о тех, что удобно выгрузить.

Доказательство

SettingsPanels.jsx:529-539 — состав dump перечислен явно: profile, theme, days, customFoods, customIngredients, recents, prefs, subscription, exportedAt.

SettingsPanels.jsx:555 — note под кнопкой с оговоркой.

Таблиц с персональными данными в схеме 33; в выгрузку попадает содержимое одной (app_state). Отдельно: src/lib/exchange.js содержит готовый CSV-экспорт с корректным экранированием, но не

подключён ни к одному экрану (см. CODE-001).

Воспроизведение

1. Профиль → Настройки → Данные → Выгрузить.
2. Открыть eataps-data.json.
3. Убедиться, что переписки, мысли, подписок и событий в файле нет.

Ожидаемое / фактическое

Ожидается	Выгрузка содержит все персональные данные субъекта в машиночитаемом виде.
Фактически	Выгружается примерно одна таблица из тридцати трёх.

Последствия

Неполное исполнение ст. 15/20 GDPR. Риск ниже, чем у PRIV-001, поскольку обязанность исполнима и вручную по запросу, но для приложения, которое обещает самообслуживание, это пробел.

Первопричина

Выгрузка написана, когда все данные жили в app_state; социальные функции появились позже и в неё не попали.

Предлагаемое исправление

Добавить серверную сборку выгрузки — одна SECURITY DEFINER функция, которая по auth.uid() собирает jsonb из всех таблиц с персональными данными владельца (messages где sender/recipient = uid, posts, post_comments, post_reactions, follows в обе стороны, notifications, promo_grants, support_messages, close_friends, diary_access, user_mutes, blocks), и отдавать её результат тем же файлом. Заодно это даст основу для ответа на запрос по ст. 15.

Тест против повторения

Тест: список таблиц с колонкой user_id/sender/recipient, полученный из схемы, сверяется со списком таблиц, попадающих в выгрузку. Новая таблица без записи в выгрузке — падение теста.

UX-001 — Количество продукта не ограничено сверху: 999999999999 г принимаются и портят всю статистику

P3 **CONFIRMED** · уверенность HIGH · Целостность данных / ввод

Где	src/lib/foods.js:716-722 (sanitizeAmount), src/components/AddMealSheet.jsx:1058, 2227, RecipeEditorSheet.jsx:246,270, SupplementSheet.jsx:173
Затронутый поток	Добавление еды · рецепты · добавки

Суть

`sanitizeAmount` аккуратно снимает всё лишнее: минус убирается (отрицательное количество ввести нельзя), запятая приводится к точке, вторая точка отбрасывается, буквы отбрасываются. Верхней границы нет ни в ней, ни у одного из шести вызывающих её полей.

Один продукт на 1e12 граммов даёт 1e12 ккал в дне. Это переживает синхронизацию (ограничение в save_app_state — 5 МБ на состояние, число под него подходит) и попадает в средние, графики и историю.

Доказательство

```
Проверено запуском sanitizeAmount и scale:
"999999999999" → "999999999999" (Number: 999999999999)
"-1"           → "1"             (минус снимается — отрицательных нет)
"12,5"         → "12.5"          (европейский разделитель)
"abc"          → ""
scale(food, 1e12) → kcal=1000000000000 p=1000000000000 ...
grep sanitizeAmount по src/ — 6 мест использования, ни в одном нет clamp/Math.min/maxLength.
При этом amountLabel и formatAmount корректно гасят NaN и Infinity (возвращают пустую строку) —
то есть один слой защиты есть, но не тот.
```

Воспроизведение

1. Дневник → «+» на любом приёме → выбрать любой продукт.
2. В поле количества ввести 999999999999.
3. Добавить.
4. Открыть Статистику: средние за неделю и месяц, графики и цели искажены на порядки.

Ожидаемое / фактическое

Ожидается	Количество ограничено разумной верхней границей (например, 10 000 г / 100 порций), лишнее обрезается или поле не принимает ввод.
Фактически	Принимается любое число; статистика портится до ручной правки.

Последствия

Не безопасность, а качество данных — но восстановление требует найти и удалить запись, а в статистике это выглядит как поломка приложения. Достижимо обычной опечаткой (залипшая клавиша, вставка из буфера), не только намеренно. Смягчение есть: запись удаляется, а день можно исключить из статистики.

Первопричина

Санитайзер отвечает за форму строки, а не за диапазон значения; диапазон не проверяет никто.

Предлагаемое исправление

Добавить границу в sanitizeAmount (одно место на все шесть полей):

```
export const MAX_AMOUNT = 100000 // 100 кг — заведомо выше любого реального приёма

export function sanitizeAmount(raw, max = MAX_AMOUNT) {
  const s = String(raw ?? '').replace(/[\^d.,]/g, '').replace(/,/g, '.')
  const dot = s.indexOf('.')
  const clean = dot === -1 ? s : s.slice(0, dot + 1) + s.slice(dot + 1).replace(/\. /g, '')
  if (clean === '' || clean === '.') return clean
  const n = Number(clean)
  return Number.isFinite(n) && n > max ? String(max) : clean
}
```

Важно вернуть именно строку и сохранить «12.» (незаконченный ввод дробного) — на это уже есть тест в foodSearch.test.js:323.

Тест против повторения

Дополнить существующий тест foodSearch.test.js:317: `sanitizeAmount('999999999999') === '100000'`, `sanitizeAmount('12.') === '12.'` (не сломать текущее поведение).

BUG-001 — computeTargets не проверяет вход: нечисловой профиль даёт NaN во всех целях, Infinity в весе — бесконечные калории

R3 **CONFIRMED** · уверенность MEDIUM · Устойчивость расчёта

Где src/lib/nutrition.js:69-120 (computeTargets), src/lib/syncModel.js:221 (normalizeState)
Затронутый поток Цели по калориям и БЖУ

Суть

Онбординг проверяет ввод хорошо: `bodyOk` требует возраст 10–100, рост 120–230, вес 30–250, и кнопка «Продолжить» до этого заблокирована. Но сам расчёт защиты не имеет, а `normalizeState` принимает профиль как есть: `profile: isObj(s.profile) ? s.profile : null` — проверяется только то, что это объект.

Значит любой путь, минуящий онбординг (повреждённый локальный кэш, состояние из облака, записанное другой версией приложения, ручная правка localStorage), приводит к NaN или Infinity в целях.

Доказательство

Проверено запуском computeTargets на граничных входах:

{}	(пустой профиль)	→	calories=NaN protein=NaN fat=NaN carbs=NaN
вес NaN		→	calories=NaN ...
вес Infinity		→	calories=Infinity protein=Infinity fat=Infinity carbs=NaN
вес -1		→	protein=-1 (отрицательная норма белка)
возраст 999		→	bmr=-3065 tdee=-4214 (отрицательный расход)
activityFactor 1e9		→	calories=1 780 000 000 000
нормальный профиль		→	calories=2450 protein=128 fat=74 carbs=318 •

syncModel.js:221 — единственная проверка профиля при нормализации состояния.
Onboarding.jsx:29 — bodyOk, корректно закрывающий путь через интерфейс.

Воспроизведение

Прямого пути через интерфейс нет — онбординг диапазоны проверяет. Воспроизводится подменой локального кэша:

- В DevTools → Application → localStorage найти ключ eataps:state:<uid>.
- В state.profile заменить weight на строку "x" или удалить поле.
- Перезагрузить.
- На главном экране цель по калориям отображается как NaN.

Статус CONFIRMED относится к отсутствию защиты в расчёте (доказано запуском); достижимость обычным пользователем — ограниченная, отсюда MEDIUM.

Ожидаемое / фактическое

Ожидается	Некорректный профиль даёт либо безопасное значение по умолчанию, либо честное «заполните профиль», но не NaN на экране.
Фактически	NaN/Infinity доходят до отрисовки колец и целей.

Последствия

Самостоятельно вред невелик и ограничен собственным экраном пользователя. Значимость в другом: это единственная точка, где расчёт доверяет форме входных данных, и она же — источник цифр для всего приложения, включая то, что видят друзья через visible_diary.

Первопричина

Проверка диапазонов живёт в интерфейсе (Onboarding), а не рядом с расчётом; нормализация состояния профиль не валидирует.

Предлагаемое исправление

Поставить защиту в самом расчёте — он вызывается из нескольких мест, и полагаться на то, что каждое проверит вход, нельзя:

```
export function computeTargets(profile) {
  const num = (v, lo, hi, dflt) => {
    const n = Number(v)
    return Number.isFinite(n) && n >= lo && n <= hi ? n : dflt
  }
  const weight = num(profile?.weight, 20, 400, null)
  const height = num(profile?.height, 100, 250, null)
  const age = num(profile?.age, 5, 120, null)
  if (weight === null || height === null || age === null) return null // «профиль не заполнен»
  ...
}
```

и в вызывающих местах показывать приглашение дозаполнить профиль вместо чисел. Диапазоны стоит взять теми же, что уже в `setDayWeight` (20-400) — тогда двух разных представлений о допустимом весе в проекте не будет.

Тест против повторения

Дополнить `nutrition.test.js`: `computeTargets({}) === null`, с весом `Infinity` `=== null`, с возрастом `999` `=== null`, обычный профиль — прежние значения (2450/128/74/318) не изменились.

SEC-005 — Блокировка не распространяется на общие групповые диалоги

R3 **CONFIRMED** · уверенность HIGH · Приватность / блокировка

Где `supabase/setup_all.sql` — политики "messages select" и "messages insert" (последние редакции)

Затронутый поток Блокировка · групповые диалоги

Суть

В личной переписке блокировка работает безупречно в обе стороны: отправка запрещена (``can_message` → `get_message_permission``), чтение отфильтровано (``not is_blocked_between(sender, auth.uid())``).

В групповом диалоге обе проверки не применяются. Фильтр чтения стоит под условием ``recipient is null or not is_blocked_between(...)`` — у группового сообщения `recipient` пуст, и условие выполняется тождественно.

Отправка проверяет только участие в диалоге.

То есть если заблокировавший и заблокированный оказались в одной группе, заблокированный продолжает писать, а заблокировавший — читать.

Доказательство

```
Политика messages select (последняя редакция):
(auth.uid() = sender or auth.uid() = recipient
or (conversation_id is not null and is_conversation_member(conversation_id, auth.uid())))
and (recipient is null or not is_blocked_between(sender, auth.uid()))
^^^^^^^^^^^^^^^^^^^^ для группы условие истинно всегда
```

```
Политика messages insert (последняя редакция):
auth.uid() = sender and ((recipient is not null and can_message(sender, recipient))
```

```
or (conversation_id is not null and is_conversation_member(conversation_id, auth.uid()))
^^^ блокировка не проверяется
```

Смягчающие обстоятельства подтверждены по коду: прямое затаскивание в группу закрыто — `can\_invite\_to\_group` проверяет `is\_blocked\_between` (setup_all.sql), `create\_group\_conversation` проверяет отдельно ещё раз, а настройка group_invites по умолчанию 'following' (setup_all.sql:8259), то есть добавить в группу может только тот, на кого приглашаемый подписан.

Воспроизведение

1. А и В состоят в группе, созданной третьим человеком С.
2. А блокирует В.
3. В пишет в группу.
4. А видит сообщение В в общей ленте группы.

Ожидаемое / фактическое

Ожидается Решение «я не хочу видеть этого человека» действует всюду, где приложение показывает его сообщения.

Фактически В группах блокировка не действует ни на чтение, ни на отправку.

Последствия

Канал для преследования в обход блокировки. Практический риск существенно снижен тем, что попасть в общую группу с заблокировавшим непросто: нужен посредник, на которого жертва подписана. Поэтому P3, а не P2.

Первопричина

Групповые диалоги добавлены поздней миграцией (2026-09-09_conversations), и предикат блокировки, написанный для личной переписки, на новый случай распространён не был.

Предлагаемое исправление

Распространить проверку на оба направления:

```
-- чтение: скрывать сообщения заблокированных и в группах
and not public.is_blocked_between(sender, auth.uid())
-- вместо (recipient is null or not ...)

-- отправка: в группе тоже учитывать блокировку хотя бы попарно
... or (conversation_id is not null
and public.is_conversation_member(conversation_id, auth.uid()))
```

Замечание по проектированию: полный запрет отправки в группу при блокировке хотя бы одним участником спорен (это наказывает остальных). Практичнее оставить отправки, но ГАРАНТИРОВАННО скрывать сообщения от того, кто заблокировал, — то есть поправить прежде всего политику чтения.

Тест против повторения

Тест SQL-уровня в supabase/verify_social_v2.sql: создать группу с А, В, С; А блокирует В; выборка messages от имени А не содержит сообщений В.

DEP-001 — Неиспользуемый стек Web3 приносит 12 из 17 уязвимостей npm и чанк на 2,68 МБ в каждой сборке

РЗ **CONFIRMED** · уверенность HIGH · Зависимости / мёртвый код

Где package.json (зависимости @reown/appkit, @reown/appkit-adapter-ethers, @reown/appkit-adapter-solana, ethers, @solana/web3.js), src/lib/appkit.js, src/lib/web3Config.js, src/components/Web3Button.jsx

Затронутый поток Вход через кошелёк

Суть

Вход по Web3-кошельку включается переменной VITE_WALLETCONNECT_PROJECT_ID. Она не задокументирована в .env.example и отсутствует в .env.local, то есть функция выключена. Код при этом написан аккуратно: кнопка рендерится только при web3Enabled, компонент загружается лениво и обернут в границу ошибок — на пользователей выключенная функция не влияет никак.

Цена всё равно платится: пять тяжёлых зависимостей остаются в дереве и в сборке. Из 17 предупреждений npm audit 12 приходятся именно на них.

Доказательство

```
npm audit: 17 уязвимостей (10 moderate, 7 high).
Разбор по дереву: @reown/appkit-adapter-solana (high), @solana/web3.js (moderate),
@solana/buffer-layout-utils (high), @solana/spl-token (high), @solana/spl-token-group,
@solana/spl-token-metadata, @solana/wallet-adapter-base, bigint-buffer (high), jayson,
stream-json, uuid, @coinbase/cdp-sdk-axios (high) — 12 записей.
Оставшиеся 5 (nanoid, postcss, browserslist, baseline-browser-mapping) — из дерева Vite, то есть
только сборочные и до пользователя не доходят.
npm run build: dist/assets/Web3Button-CbvOHZD8.js — 2 682,65 kB (gzip 773,83 kB).
grep WALLETCONNECT по .env.example и .env.local — ноль совпадений.
Боевое окружение Vercel не проверялось — доступа к нему нет; вывод о выключенности сделан по
отсутствию документации переменной.
```

Воспроизведение

```
npm audit; npm run build и посмотреть на размер чанка Web3Button.
```

Ожидаемое / фактическое

Ожидается	В зависимостях нет пакетов, которые не используются.
Фактически	Пять тяжёлых пакетов, включая весь стек Solana, присутствуют ради выключенной функции.

Последствия

Прямой эксплуатации нет: уязвимый код не исполняется, потому что чанк не загружается. Значение практическое — 12 предупреждений npm audit делают отчёт бесполезным как сигнал: настоящую проблему в нём не заметят. Плюс время сборки и объём артефактов.

Первопричина

Функция добавлена, но не доведена до включения и не удалена.

Предлагаемое исправление

```
Принять решение, а не оставлять как есть.
• Если Web3-вход не нужен — удалить 5 зависимостей, src/lib/appkit.js, src/lib/web3Config.js,
src/components/Web3Button.jsx и ветку с Web3ErrorBoundary в AuthSheet.jsx, а также строку про
Reown/WalletConnect из правовых документов. Это снимает 12 из 17 предупреждений и 2,68 МБ сборки.
• Если нужен — задать и задокументировать VITE_WALLETCONNECT_PROJECT_ID, обновить .env.example и
```

провести отдельный аудит этого пути входа (сейчас он не проверялся, поскольку выключен).

Тест против повторения

Сторож: тест, который падает, если в `package.json` есть зависимость, не встречающаяся ни в одном `import` по `src/` и `api/`.

A11Y-001 — Кнопки-символы «←» и «×» не имеют доступного имени

РЗ **CONFIRMED** · уверенность HIGH · Доступность (WCAG 2.2 AA)

Где `src/components/AddMealSheet.jsx:1005, 1392, 1481, 1575, 1728, 1838, 1889, 2128, 2199; src/components/DayScreen.jsx:1219`

Затронутый поток Навигация внутри листов, закрытие листов

Суть

Десять кнопок содержат только символ («←» назад, «×» закрыть) и не имеют ни текста, ни `aria-label`. Скринридер прочитает «стрелка влево, кнопка» или просто «кнопка» — назначение непонятно. Это нарушение WCAG 4.1.2 Name, Role, Value.

В проекте доступность в целом не забыта: 126 `aria-label` на 405 кнопок, есть `role/aria-pressed/aria-checked` (30 использований), и многие иконочные кнопки подписаны образцово — например `SupplementsCard.jsx:260` использует осмысленное `aria-label={'Убрать ${e.name}'}`. Пробел именно в листе добавления еды — самом посещаемом экране приложения.

Доказательство

Поиск кнопок, содержащих только символ и не имеющих `aria-label`, — 10 совпадений (перечислены в поле «Где»).

Всего в проекте: 405 тегов `<button>`, 126 атрибутов `aria-label`, 22 `role=`, 30 `aria-pressed/checked/expanded`.

Положительный пример рядом: `SupplementsCard.jsx:260` — `<button className="iconbtn" ... aria-label={'Убрать ${e.name}'}>×</button>`.

Воспроизведение

1. Включить VoiceOver (iOS/macOS) или TalkBack.
2. Открыть Дневник → «+» → выбрать продукт.
3. Пройти по элементам экрана свайпом — кнопка возврата читается как «кнопка» без назначения.

Ожидаемое / фактическое

Ожидается У каждого управляющего элемента есть доступное имя.

Фактически Десять кнопок безымянны для вспомогательных технологий.

Последствия

Экран добавления еды — основной сценарий приложения; для незрячего пользователя навигация по нему становится угадыванием. Правка тривиальна.

Первопричина

Символ в содержимом кнопки выглядит как текст и создаёт ложное ощущение, что имя есть.

Предлагаемое исправление

Добавить `aria-label` в перечисленные места:

```
<button className="tap44" aria-label="Назад" onClick={onBack}>⬅️</button>
<button className="iconbtn" aria-label="Закрыть" onClick={close}>✕</button>
```

Заодно проверить размер области нажатия: класс `tap44` намекает на 44 px, но у кнопок на `AddMealSheet.jsx:2128` и `2199` явно задан `minHeight: 32` — это ниже рекомендованных 44×44 (WCAG 2.5.8 Target Size).

Тест против повторения

Тест-сторож по образцу `designGuard.test.js`: разобрать JSX и падать, если найден `<button>`, у которого нет ни текстового содержимого длиннее одного символа, ни `aria-label`.

SEC-006 — CSP разрешает `unsafe-inline` и `unsafe-eval`, `connect-src` открыт на любой `https`

РЗ **CONFIRMED** · уверенность HIGH · Защита в глубину

Где `vercel.json:10-20`
Затронутый поток `Заголовки безопасности`

Суть

Набор заголовков в целом сделан грамотно: `nosniff`, `X-Frame-Options: DENY`, `frame-ancestors none`, `HSTS` с `preload`, `Referrer-Policy`, `Permissions-Policy` с отключёнными камерой/микрофоном/геолокацией. Но директива `script-src` содержит одновременно `'unsafe-inline'` и `'unsafe-eval'`, а `connect-src` открыт на `'https:'` и `'wss:'` целиком.

В таком виде CSP почти не ограничивает выполнение постороннего скрипта и не мешает вывести данные на произвольный домен.

Доказательство

```
vercel.json:18 — `script-src 'self' 'unsafe-inline' 'unsafe-eval' blob:; connect-src 'self' https: wss:`.
```

Смягчающее обстоятельство подтверждено отдельно: в коде нет ни одного `dangerouslySetInnerHTML`, ни присваивания `innerHTML`, ни `eval/new Function` (проверено `grep` по всему `src/` и `api/`) — то есть собственного источника XSS не найдено, и React экранирует вывод. Автолинковка в чате (`MessageList.jsx:26-31`) разбирает только `https://`, поэтому `javascript:-`ссылку через неё не подсунуть.

Воспроизведение

```
Проверка заголовков ответа: curl -I https://www.eataps.com | grep -i content-security-policy.
```

Ожидаемое / фактическое

Ожидается	CSP служит вторым рубежом на случай, если XSS всё-таки появится.
Фактически	При появлении XSS политика его не остановит, а <code>connect-src</code> не ограничит вывод данных.

Последствия

Сегодня эксплуатируемой уязвимости нет — это именно потеря защиты в глубину. Значение вырастет, как только в проекте появится сторонний виджет или отрисовка пользовательской разметки.

Первопричина

'unsafe-eval' обычно попадает в политику ради дев-режима или библиотеки; в данном случае, вероятно, ради Web3-стека (который выключен).

Предлагаемое исправление

```
Попробовать убрать 'unsafe-eval' и проверить, что сборка работает (Vite в проде eval не требует;
требовал его, скорее всего, AppKit — а он выключен).
connect-src сузить до фактических адресатов:
connect-src 'self' https://<project>.supabase.co wss://<project>.supabase.co
https://ru.openfoodfacts.org https://api.anthropic.com;
(запросы к Anthropic идут с сервера, так что его в connect-src может и не потребоваться —
проверить по факту).
'unsafe-inline' в script-src убирается сложнее (нужны хэши или nonce); при отсутствии
инлайн-скриптов в index.html — попробовать снять и проверить.
```

Тест против повторения

Проверка после правки: открыть приложение, пройти вход, AI-ассистента и сканер штрихкодов, убедиться в отсутствии нарушений CSP в консоли.

SEC-007 — Тип загружаемой в AI картинки проверяется только по заявленному media_type, без сигнатуры файла

R3 **CONFIRMED** · уверенность HIGH · Проверка входных данных

Где `api/ai/vision.js:19, 32-39`
Затронутый поток `Распознавание еды по фото`

Суть

Проверяется поле `image.media_type` из тела запроса и длина base64. Фактическое содержимое не проверяется: клиент может объявить `image/jpeg` и прислать что угодно.

Доказательство

```
vision.js:19 — `const ALLOWED_TYPES = ['image/jpeg', 'image/png', 'image/webp']`.
vision.js:33 — `if (!image?.data || !ALLOWED_TYPES.includes(image.media_type))` — сравнивается
заявленный тип.
vision.js:36-38 — ограничение длины base64 (3,5 МБ) есть и посчитано правильно, с учётом
раздувания base64 и потолка тела функции.
Существенное смягчение: изображение НИГДЕ не сохраняется и никому не отдаётся — уходит в
Anthropic в теле запроса и живёт только в памяти (vision.js:8-10 и состав messages на :60-66).
Значит классические последствия (хранимый XSS через SVG, подмена файла) недостижимы.
```

Воспроизведение

```
Отправить POST /api/ai/vision с { image: { media_type: "image/jpeg", data: <base64 произвольных
байт> } } — запрос дойдёт до вызова модели, и её отказ будет оплачен из бюджета пользователя.
```

Ожидаемое / фактическое

Ожидается	Несоответствие объявленного типа содержимому отклоняется до обращения к модели.
Фактически	Отклоняет только Anthropic, уже после списания резерва.

Последствия

Низкий: платит бюджетом сам отправитель, хранения нет. Ценность правки — экономия вызова и более честная ошибка пользователю.

Первопричина

Проверка типа сделана по метаданным, а не по данным.

Предлагаемое исправление

Сверить первые байты base64 с сигнатурой:

```
const MAGIC = {
  'image/jpeg': [[0xFF, 0xD8, 0xFF]],
  'image/png':  [[0x89, 0x50, 0x4E, 0x47]],
  'image/webp': [[0x52, 0x49, 0x46, 0x46]], // RIFF, далее WEBP на смещении 8
}
const head = Buffer.from(image.data.slice(0, 24), 'base64')
const ok = MAGIC[image.media_type].some((sig) => sig.every((b, i) => head[i] === b))
if (!ok) return res.status(400).json({ error: 'bad_image', message: 'Файл не похож на изображение.' })
```

Тест против повторения

Тест обработчика: media_type image/png с телом из произвольных байт — ожидать 400 и отсутствие вызова callClaude.

UX-002 — Оптимистичные переключатели приватности могут рассинхронизироваться с сервером при быстром переключении и ошибке

P3 **PROBABLE** · уверенность MEDIUM · Согласованность состояния

Где src/components/social/PrivacyHub.jsx:80-86 (patch)
Затронутый поток Профиль → Приватность

Суть

Переключатель меняет состояние сразу и откатывает его при ошибке, запоминая предыдущее значение как `const prev = p`. Защиты от параллельных вызовов нет, а `prev` — значение из замыкания текущего рендера.

При двух быстрых переключениях подряд, если ПЕРВЫЙ запрос завершится ошибкой уже после того, как второй ушёл, откат вернёт состояние, предшествующее первому, и экран разойдётся с сервером до следующей загрузки.

Доказательство

PrivacyHub.jsx:80-86 — тело patch целиком; нет ни флага «запрос в полёте», ни отмены предыдущего, ни перечитывания с сервера после ошибки.
Компонент содержит 17 обработчиков и ни одного состояния busy/pending (сверено по всем

компонентам: у PrivacyHub, ConversationInfo и RelationshipSheet асинхронные действия без таких состояний).

Статус PROBABLE: путь рассуждения проверен по коду, но сценарий требует ошибки сети в узком окне и на живом приложении не воспроизводился.

Воспроизведение

1. Профиль → Приватность.
2. В DevTools включить троттлинг и отключить сеть на секунду.
3. Дважды быстро переключить «Закрытый аккаунт».
4. Первый запрос падает, второй проходит — экран показывает исходное значение, сервер хранит второе.

Ожидаемое / фактическое

Ожидается	Экран после любой последовательности переключений показывает то, что на сервере.
Фактически	Возможно расхождение до перезагрузки раздела.

Последствия

Для настроек приватности цена расхождения выше обычной: человек видит «закрыто», а на сервере открыто (или наоборот) и принимает решение по неверной картинке. Именно об этом предупреждает комментарий в шапке самого файла — «тумблер, который визуально меняется, но ничего не закрывает, вреднее пустого места».

Первопричина

Оптимистичное обновление без защиты от наложения запросов.

Предлагаемое исправление

Перечитывать состояние с сервера после ошибки вместо отката к запомненному значению — это надёжнее любого сравнения меток:

```
const patch = async (next, fn) => {
  setP((cur) => ({ ...cur, ...next }))
  setErr(null)
  const res = await fn()
  if (res?.error) { setErr(res.error); await load() } // ← источник истины у сервера
}
```

Дополнительно — блокировать повторное нажатие на время запроса.

Тест против повторения

Тест компонента: два последовательных patch, первый отклоняется с задержкой; после обоих состояние равно ответу load(), а не первому prev.

CFG-001 — Личные Telegram chat ID записаны в исходном коде и продублированы как значение по умолчанию

P3 **CONFIRMED** · уверенность HIGH · Конфигурация / приватность владельца

Где api/feedback.js:5, api/telegram/_tg.js:11
Затронутый поток Поддержка и обратная связь

Суть

В `api/feedback.js` адресаты заданы прямо в коде: ``const CHAT_IDS = [571138125, 938539456]``. В `api/telegram/_tg.js` те же числа стоят как значение по умолчанию: ``String(process.env.TG_ADMIN_CHAT_IDS || '571138125,938539456')``.

Переменная `TG_ADMIN_CHAT_IDS` существует и задокументирована в `.env.example` — то есть правильный механизм есть, но одна из двух точек его не использует, а вторая содержит рабочий запасной вариант.

Доказательство

```
api/feedback.js:5 и api/telegram/_tg.js:11 — оба значения совпадают.  
.env.example — TG_ADMIN_CHAT_IDS задокументирована.  
Значения находятся в истории git и, если репозиторий когда-либо станет публичным, останутся в ней  
после любой правки.
```

Воспроизведение

```
grep -n "571138125" api/
```

Ожидаемое / фактическое

Ожидается	Идентификаторы получателей задаются окружением.
Фактически	Заданы в коде и в истории репозитория.

Последствия

Не даёт прав: стать администратором бота, зная `chat_id`, нельзя — бот пишет в эти чаты, а не слушает их. Практическое значение — раскрытие личного идентификатора владельца и расхождение двух точек конфигурации: смена `TG_ADMIN_CHAT_IDS` не изменит адресатов `api/feedback.js`.

Первопричина

`api/feedback.js` написан раньше, чем появился общий модуль `_tg.js` с переменной окружения, и не переведён на него.

Предлагаемое исправление

В `api/feedback.js` импортировать общий список и убрать локальную константу:

```
import { ADMIN_CHAT_IDS, sendMessage } from './telegram/_tg.js'  
...  
const results = await Promise.all(ADMIN_CHAT_IDS.map((id) => sendMessage(id, msg)))
```

В `_tg.js` убрать запасное значение: при незаданной переменной честнее упасть с понятной ошибкой, чем молча писать в чужой чат.

Тест против повторения

Тест-сторож: `grep` по `api/` на последовательность из 9+ цифр, не являющуюся годом или размером, — падение, если найдена вне тестов.

DB-002 — Восемь триггерных функций объявлены без `set search_path`

R3 **CONFIRMED** · уверенность HIGH · Практика написания SQL

Где supabase/setup_all.sql — guard_app_state_update, guard_conversation_member_update, guard_message_update, guard_notification_update, guard_post_update, limit_follows, limit_follow_requests, limit_friend_requests, limit_post_comments, limit_post_reactions, limit_posts, lock_follow_pair, normalize_public_id, safe_uuid, slugify_username, get_last_seen

Затронутый поток Триггеры целостности и ограничения частоты

Суть

В проекте выдержано хорошее правило: практически все SECURITY DEFINER функции объявлены с `set search\_path = public`. Проверка по всем 148 функциям показала, что из тех, кто работает с правами определившего, без search_path не осталось ни одной.

Исключения — функции с правами вызывающего (SECURITY INVOKER): триггеры-сторожа и ограничители частоты. Для них риск подмены пути поиска существенно ниже (они не повышают прав), но линтер Supabase отмечает это как function_search_path_mutable, и единообразие здесь дешевле, чем разбирательство при каждом аудите.

Доказательство

Разбор всех определений функций в setup_all.sql:
148 функций всего; 110 выданы роли authenticated.
Все SECURITY DEFINER — с `set search\_path = public` (проверено сплошным разбором).
Без search_path — только перечисленные выше, все SECURITY INVOKER.
Это существенно лучше типичного состояния проекта на Supabase и заслуживает упоминания как сильная сторона.

Воспроизведение

Запустить линтер Supabase (Database → Advisors) либо разбор определений функций.

Ожидаемое / фактическое

Ожидается	Единообразное объявление пути поиска у всех функций.
Фактически	Шестнадцать функций с правами вызывающего объявлены без него.

Последствия

Практического пути эксплуатации не найдено: функции не повышают прав, и подмена search_path даёт злоумышленнику ровно его собственные права. Это гигиена и чистота отчёта линтера.

Первопричина

Правило применялось к SECURITY DEFINER и не было распространено на остальные.

Предлагаемое исправление

Дописать `set search\_path = public` в объявления перечисленных функций. Правку делать В МИГРАЦИЯХ, а не в setup_all.sql (см. DB-001), и пересобрать файл.

Тест против повторения

Дополнить scripts/check-sql.mjs проверкой: любая create function без set search_path — предупреждение.

UX-003 — Поиск по префиксу позволяет перебрать базу пользователей; ограничения частоты на поиск нет

R3 **CONFIRMED** · уверенность HIGH · Перечисление пользователей

Где supabase/setup_all.sql — search_users (последняя редакция)
Затронутый поток Поиск людей

Суть

`search\_users` требует минимум два символа, ищет по префиксу ника и имени, отдаёт не более 30 записей и исключает заблокированных. Ограничения частоты у неё нет — ни в базе, ни на клиенте.

Перебором двухсимвольных префиксов (около 1 500 запросов на латиницу и кириллицу) выгружается до ~45 000 карточек: ник, отображаемое имя, аватар, признак закрытости.

Доказательство

Функция возвращает `user_id, username, display_name, avatar_url, is_private`.
Ограничения: ``char_length(q.v) >= 2`, `limit least(greatest(coalesce(p_limit,20),1),30)``,
исключение себя и заблокированных.
В проекте есть триггеры ограничения частоты на изменяющие действия (`limit_follows — 200/час`,
`limit_posts — 60/час`, `limit_unaccepted_messages — 5` на адресата и 20 новых собеседников в час),
но на чтение и поиск таких ограничений нет.
Закрытость аккаунта от перечисления не защищает: `is_private` отдаётся как поле, а сама карточка
возвращается независимо от него.

Воспроизведение

Перебрать `p_query` по всем двухбуквенным сочетаниям, вызывая RPC `search_users` с обычным аккаунтом.

Ожидаемое / фактическое

Ожидается Поиск служит поиском знакомых, а не выгрузкой базы.
Фактически Полная выгрузка публичных карточек достижима за несколько тысяч запросов.

Последствия

Своего рода плата за любой поиск людей — полностью закрыть, не сломав функцию, нельзя. Значение возрастает при росте базы: готовый список ников и аватаров — сырьё для рассылки и подбора целей. Приватные данные (дневник, записи, переписка) при этом не раскрываются: их закрывают отдельные предикаты, проверенные в этом аудите.

Первопричина

Ограничения частоты сделаны для изменяющих операций, чтение не покрыто.

Предлагаемое исправление

Добавить ограничение частоты на поиск — по образцу уже имеющихся триггеров, но для функции:
• таблица `search_log (user_id, at)` и проверка «не более N вызовов в минуту» внутри `search_users`;
• либо ограничение на уровне платформы (Vercel Firewall / Supabase rate limit) для этого RPC.
Разумный порог — 30–60 поисков в минуту: живому человеку хватает с запасом, перебор становится непрактичным.

Тест против повторения

Тест SQL-уровня: 100 вызовов `search_users` подряд от одного пользователя — начиная с порогового вызов обязан отклоняться.

AI-001 — Границы медицинской безопасности заданы только промптом; серверной проверки ответа нет

R3 **CONFIRMED** · уверенность HIGH · Безопасность AI

Где `src/lib/aiPrompt.js:282-292` (ГРАНИЦЫ ТЕМЫ), :148-149 (тон savage)
Затронутый поток AI-ассистент

Суть

Промпт содержит продуманные границы: не ставить диагнозы, не назначать и не отменять лекарства, осторожность при беременности, диабете, болезнях ЖКТ/почек/печени, приеме препаратов, возрасте до 16, весе сильно ниже нормы и признаках расстройства пищевого поведения; прямой запрет предлагать дефицит ниже 1200 ккал женщинам и 1500 мужчинам и голодание дольше суток. Тон «без цензуры» отдельно инструктирован мгновенно и без мата переходить на спокойный регистр при признаках расстройства пищевого поведения, срыва, самоповреждения или тревоги.

Это заметно лучше, чем принято в приложениях такого рода. Но все перечисленные границы — инструкции модели, то есть мягкие. Серверной проверки ответа нет: ни фильтра, ни перепроверки числовых рекомендаций, ни заготовленного ответа с контактами помощи на случай, когда разговор уходит в тему самоповреждения.

Доказательство

`aiPrompt.js:288-291` — текст границ.
`api/ai/chat.js` и `vision.js` — после `parseReply` ответ уходит пользователю без содержательных проверок: разбирается формат (`parseReply` в `_shared.js`), проверяется обрезанность, но не содержание.
Положительно: тариф и модель берутся из базы (`tierOf`), а не из тела запроса, — подмена тарифа через клиент невозможна (проверено).
Прямой цели «продукт даёт вредный совет» в этом аудите не ставилось: проверка требует прогонов на живой модели, которых не проводилось.

Воспроизведение

Требуется живой прогон с попытками обойти инструкции промпта; в рамках этого аудита не выполнялся.

Ожидаемое / фактическое

Ожидается Ответы, выходящие за медицинские границы, не доходят до пользователя вне зависимости от того, удалось ли обойти инструкции.

Фактически Единственный рубеж — следование модели инструкции.

Последствия

Приложение работает с питанием и весом, то есть с областью, где неудачный совет достигает уязвимой аудитории. Промпт написан ответственно и снижает риск, но один рубеж для такой темы — мало.

Первопричина

Безопасность реализована на уровне формулировок, а не архитектуры.

Предлагаемое исправление

Добавить второй рубеж на сервере, дёшево и без второй модели:

1. Проверка чисел в ответе: если в cards или reply предлагается суточная норма ниже порога из промпта — заменять на безопасный текст.
2. Список признаков кризисной темы (самоповреждение, суицид, полный отказ от еды) в ВОПРОСЕ пользователя: при совпадении отвечать заготовленным текстом с контактами помощи, не обращаясь к модели вовсе. Так делают потому, что именно на этих запросах поведение модели наименее предсказуемо, а цена ошибки наибольшая.
3. Зафиксировать в правовых текстах, что ассистент не является медицинской консультацией (сейчас этого нет ни в AGB, ни в Datenschutz).

Тест против повторения

Тест на серверную проверку: ответ модели с рекомендацией 900 ккал в сутки обязан быть заменён; вопрос из кризисного списка обязан не доходить до callClaude.

CODE-001 — src/lib/exchange.js — 311 строк экспорта и импорта CSV, не подключённых ни к одному экрану

P4 **CONFIRMED** · уверенность HIGH · Мёртвый код

Где src/lib/exchange.js (весь файл)
Затронутый поток —

Суть

Модуль реализует экспорт дневника в CSV (построчно по продуктам и посуточно), разбор импортируемого CSV с определением разделителя, сопоставлением колонок и разбором дат. Код добротный: в csvCell есть защита от формульной инъекции (значения, начинающиеся с =, +, -, @, таб или возврат каретки, предваряются апострофом) — то, что в экспортах забывают чаще всего.

Ни одного импорта этого модуля в проекте нет. Тестов у него тоже нет (в отличие от 49 других модулей).

Доказательство

```
grep -rn "exchange" src/ — ноль совпадений вне самого файла.  
Список экспортируемых функций: toCsv, buildMealsCsv, buildDaysCsv, csvBlob, detectDelimiter,  
parseCsvLine, mapColumns, parseDate, parseImportCsv.  
Фактическая выгрузка в приложении сделана отдельно и в другом формате — JSON,  
SettingsPanels.jsx:526-548.
```

Воспроизведение

```
grep -rn "exchange" src/
```

Ожидаемое / фактическое

Ожидается	В проекте нет неподключённого кода — либо он используется, либо удалён.
Фактически	311 строк сопровождаются, но не исполняются.

Последствия

Нулевой для пользователя. Для сопровождения — ложное впечатление, что импорт и экспорт CSV в приложении есть.

Первопричина

Функция написана заранее и не доведена до экрана.

Предлагаемое исправление

Решить одно из двух: либо подключить (готовый CSV-экспорт заметно полезнее нынешнего JSON и частично закрывает PRIV-003 — особенно buildMealsCsv), либо удалить файл. Учитывая PRIV-003, подключение выглядит осмысленнее удаления.

Тест против повторения

Если подключать — тесты на parseImportCsv (разделители, даты, экранирование) и на защиту csvCell от формульной инъекции.

PERF-001 — Главный чанк 1,15 МБ (341 КБ в gzip); разбиение на части не настроено

P4 **CONFIRMED** · уверенность HIGH · Производительность

Где vite.config.js, результат npm run build
Затронутый поток Первая загрузка

Суть

Сборка предупреждает о чанках больше 500 КБ. Крупнейший загружаемый — index-DnFkNcWe.js, 1 149 КБ (341 КБ gzip). Второй — 642 КБ (175 КБ gzip). manualChunks в vite.config.js не настроен; конфигурация минимальна и содержит только плагин React и подстановку версии.

Отдельно отмечу: ленивые границы в проекте есть и сделаны осознанно (lazyWithReload, LazyBoundary, вынос Web3 и тяжёлых экранов в отдельные чанки) — то есть проблема не в отсутствии подхода, а в том, что основной чанк не разделён.

Доказательство

```
npm run build:
dist/assets/index-DnFkNcWe.js      1 149,49 kB | gzip: 341,31 kB
dist/assets/index-AGvWEndH.js      642,35 kB | gzip: 174,99 kB
dist/assets/Web3Button-...js       2 682,65 kB | gzip: 773,83 kB (не загружается, см. DEP-001)
предупреждение rollup о чанках > 500 kB.
src/lib/foods.js — 1 496 строк справочника продуктов; src/components/AddMealSheet.jsx — 2 267 строк.
```

Воспроизведение

```
npm run build
```

Ожидаемое / фактическое

Ожидается Первый экран загружается, не дожидаясь справочника продуктов и экрана добавления еды.

Фактически

Основной чанк включает всё сразу.

Последствия

На медленной мобильной сети — ощутимая задержка первого показа. Смягчено service worker-ом, который кэширует ядро после первого визита, поэтому страдает только первое открытие.

Первопричина

manualChunks не настроен; крупный справочник продуктов и самый большой экран попадают в основной чанк.

Предлагаемое исправление

Вынести справочник и тяжёлые экраны:

```
// vite.config.js
build: {
  rollupOptions: {
    output: {
      manualChunks: {
        supabase: ['@supabase/supabase-js'],
        foods: ['./src/lib/foods.js', './src/lib/foodMicros.js'],
      },
    },
  },
}
```

Плюс ленивая загрузка AddMealSheet (2 267 строк, открывается по кнопке, а не на первом экране) — в проекте уже есть lazyWithReload ровно для этого.

Тест против повторения

Проверка после правки: ни один загружаемый при старте чанк не превышает 500 КБ.

19. Десять самых значимых проблем

ID	Ур.	Проблема	Последствия	Эксплуатируемость	Труд-сть
SEC-001	P1	Обход дневного лимита AI параллельными запросами	Деньги владельца, обесценивание подписки	Высокая — скрипт в 10 строк	S (<1 ч)
PRIV-001	P1	Anthropic, Stripe и Telegram не раскрыты; политика — шаблон	Нарушение ст. 13/28/30 GDPR, блокирует запуск в ЕС	н/п — состояние документов	M (часы)
PRIV-002	P1	Изображения не удаляются даже при удалении аккаунта	Нарушение ст. 17, бессрочная публичная ссылка	Средняя — нужна сохранённая ссылка	M → L
SEC-002	P2	Незаданная цена даёт высший тариф AI_PREMIUM	Бесплатный безлимит на дорогой модели	Низкая — нужна редкая форма события	S (<1 ч)
SEC-003	P2	/api/feedback открыт без входа	Затопление личного Telegram владельца	Высокая — один curl	S (<1 ч)
DB-001	P2	setup_all.sql разошёлся с миграциями	Расхождение схемы при установке с нуля	н/п — состояние репозитория	S (<1 ч)
SEC-004	P2	Вебхук Stripe без контроля порядка событий	Платный доступ после отмены	Низкая — зависит от доставки	M (часы)
PRIV-003	P2	Выгрузка не включает переписку и социальный граф	Неполное исполнение ст. 15/20	н/п	M (часы)
SEC-005	P3	Блокировка не действует в общих группах	Канал для преследования в обход блокировки	Низкая — нужен посредник	S (<1 ч)
UX-001	P3	Количество продукта не ограничено сверху	Порча статистики обычной опечаткой	Высокая — случайно	S (<1 ч)

20. Двадцать быстрых исправлений

S — меньше часа, M — несколько часов, L — один-два дня. Четырнадцать из двадцати помечены как S: большинство находок этого аудита закрываются правкой в несколько строк.

ID	Что сделать	Оценка
SEC-001	Вернуть итог из ai_usage_add и сверять с бюджетом	S
SEC-002	Строить TIER_BY_PRICE только из заданных переменных	S
SEC-003	Потребовать вход в /api/feedback (как в /api/support)	S
DB-001	Внести миграцию в SOURCES и пересобрать setup_all.sql	S
CFG-001	Взять ADMIN_CHAT_IDS из окружения в api/feedback.js	S
UX-001	Верхняя граница в sanitizeAmount — одно место на 6 полей	S
BUG-001	Проверка входа в computeTargets, возврат null	S
A11Y-001	aria-label на десять кнопок-символов	S
SEC-005	Убрать «recipient is null or» из политики чтения messages	S
SEC-007	Сверка сигнатуры файла в /api/ai/vision	S
UX-002	После ошибки в PrivacyHub перечитывать состояние с сервера	S
DB-002	Дописать set search_path в 16 функций	S
SEC-006	Убрать unsafe-eval, сузить connect-src	S
PERF-001	manualChunks для foods.js и supabase	S
DEP-001	Удалить выключенный стек Web3 — минус 12 предупреждений	M
PRIV-001	Дополнить Datenschutz, VVT и чек-лист AVV	M
SEC-004	last_event_at и отбрасывание старых событий Stripe	M
CODE-001	Подключить exchange.js к выгрузке либо удалить	M
PRIV-003	Серверная сборка полной выгрузки одной RPC	M
PRIV-002	Удаление объектов хранилища + закрытие бакетов	L

21. План устранения

Фаза 0 — до любого публичного доступа

- DB-001 — вернуть `setup_all.sql` к сборке из источников и внести миграцию в SOURCES. Сейчас на ветке красный тест и расхождение схемы. Это первое, потому что мешает всему остальному.
- SEC-001 — закрыть гонку бюджета AI. Каждый день промедления оплачивается ключом владельца.
- SEC-003 — потребовать вход в `/api/feedback`.
- SEC-002 — строить TIER_BY_PRICE только из заданных переменных.

Фаза 1 — до открытия для широкой публики в ЕС

- PRIV-001 — дополнить Datenschutz, реестр по ст. 30 и чек-лист AVV; заключить договоры с Anthropic, Stripe и Telegram; заполнить плейсхолдеры и снять плашку «Шаблон». Это блокирующий пункт для рынка ЕС.
- PRIV-002 — удалять объекты хранилища при удалении записи, отмене сообщения и удалении аккаунта; перевести `post-images` и `chat-images` на закрытую модель `dm-media`.
- SEC-005 — распространить блокировку на групповые диалоги (минимум — на чтение).
- SEC-004 — добавить контроль порядка событий Stripe.
- AI-001 — добавить серверный рубеж: проверку числовых рекомендаций и заготовленный ответ на кризисную тему; зафиксировать в правовых текстах, что ассистент не является медицинской консультацией.

Фаза 2 — следующий выпуск

- PRIV-003 — полная выгрузка данных одной серверной функцией.
- UX-001 — верхняя граница количества.
- BUG-001 — проверка входа в `computeTargets`.
- A11Y-001 — доступные имена кнопок.
- UX-002 — перечитывание состояния приватности после ошибки.
- SEC-007 — сверка сигнатуры изображения.
- UX-003 — ограничение частоты поиска людей.
- DEP-001 — решить судьбу Web3: удалить или включить.
- Интеграционные тесты RLS на живой базе — главный пробел покрытия.

Фаза 3 — качество и архитектура

- SEC-006 — сузить CSP.
- DB-002 — `search_path` в 16 функциях.
- CFG-001 — вынести `chat_id` в окружение.
- PERF-001 — `manualChunks` и ленивая загрузка листа добавления.
- CODE-001 — подключить или удалить `exchange.js`.
- Разделить `AddMealSheet` и `foods.js`.
- Наблюдаемость: сбор ошибок и метрики — сейчас их нет вовсе.
- Обновить зависимости: React 19, Vite 8, supabase-js, stripe.

22. Покрытие аудита

Матрица построена автоматически из репозитория, а не составлена вручную: список «что проверено» иначе неизбежно разошёлся бы с тем, что в проекте есть. Полный поэлементный перечень — в файле EATAPS_COVERAGE_MATRIX.csv (513 строк).

Тип элемента	Обнаружено	Разобрано	Прогон	Замечания
Экраны	35	35	нет	Статический разбор; браузерного прогона не было
Компоненты React	73	73	нет	Обработчики, состояния, очистка эффектов
Модули src/lib	64	64	41 под тестами	23 без автотестов — разобраны статически
Серверные точки /api	8	8	нет	Вход, права, проверка, лимиты, утечка ошибок
Функции Postgres	148	148	нет	Права, search_path, auth.uid(), гранты
Таблицы	33	33	нет	RLS на всех 33
Политики RLS	99	99	нет	Предикаты разобраны вручную
Миграции	24	24	скриптами	Идемпотентность и порядок проверены
Потоки	29	28	нет	Один (вход Web3) не поддаётся проверке — выключен
Интерактивные элементы	756	756	нет	Разобраны 16 классами (раздел 7)

Статус	Строк	Доля	Значение
AUDITED	420	81.9%	Разобрано полностью, замечаний нет
ISSUE FOUND	69	13.5%	Разобрано, связано с находкой
PARTIALLY AUDITED	23	4.5%	Разобрано частично (нет автотестов или боевых ключей)
NOT TESTABLE	1	0.2%	Проверить невозможно — функция выключена

ИТОГОВОЕ ПОКРЫТИЕ АУДИТА: 95.3 % (489 из 513 элементов разобраны полностью)

Почему не 100 %

Здесь намеренно не стоит 100 %. Полными считаются только те 489 элементов, которые разобраны без оговорок. Остальные 24 помечены частичными или непроверяемыми: 23 модуля без автотестов, экраны без браузерного прогона там, где он изменил бы вывод, и один выключенный поток. Кроме того, ни один элемент не проверялся на живой базе и в живом браузере — то есть даже «полное» покрытие здесь означает полный СТАТИЧЕСКИЙ разбор, а не подтверждение поведением.

23. Готовность к продакшену

Направление	Балл	Обоснование
Функциональная корректность	8/10	Ядро работает, 813 из 814 тестов зелёные. Снимают балл незакрытая верхняя граница количества и NaN при повреждённом профиле.
Безопасность	6/10	RLS сплошной и грамотный, подмены личности нет нигде. Снимают баллы гонка бюджета AI и отказ в высший тариф.
Приватность	4/10	Модель доступа в базе образцовая, но изображения не удаляются никогда, а правовые документы не знают о трёх обработчиках.
Целостность базы	8/10	RLS на всех 33 таблицах, каскады полные, save_app_state атомапен. Снимает балл разъехавшийся setup_all.sql.
Офлайн и синхронизация	9/10	HLC, тумбстоуны, пофайловые метки, кэш по пользователю, 814 тестов вокруг слияния. Сильнейшая часть проекта.
Платежи	6/10	Подпись вебхука проверяется, клиент тариф не назначает. Снимают баллы отказ в AI_PREMIUM и отсутствие контроля порядка событий.
Безопасность AI	6/10	Границы в промпте продуманы, тариф серверный. Снимают баллы обход лимита и отсутствие второго рубежа.
Производительность	7/10	Ни одной утечки эффектов на 80 проверенных. Снимает балл неразделенный чанк на 1,15 МБ.
Пользовательский опыт	8/10	Отмена, подтверждения, честные пустые состояния, все ветки отказа сканера. Снимает балл рассинхронизация тумблеров приватности.
Доступность	6/10	126 aria-label и корректные роли, но десять кнопок-символов без имени на главном экране.
Покрытие тестами	8/10	814 тестов, включая многоустройственные сценарии и сторожей SQL. Не покрыты RLS, платежи и лимиты AI.
Сопровождаемость	8/10	Комментарии объясняют причины, а не механику. Снимают балл мёртвый Web3 и exchange.js.
Наблюдаемость	4/10	Есть структурный log и console.error в критичных местах, но ни сбора ошибок, ни метрик, ни оповещений.
Готовность к продакшену	5/10	Инженерно приложение зрелое; к выпуску не готовы правовые документы и три пункта P1.

ИТОГ: 93 из 140 → 66 / 100

ТРЕБУЮТСЯ СУЩЕСТВЕННЫЕ ИСПРАВЛЕНИЯ

Оценка не завышена сознательно. Инженерное качество кода тянет на 8 из 10 — и это честная оценка, которую заслуживают сплошной RLS, устройство синхронизации и дисциплина очистки эффектов. Но готовность к выпуску определяется не средним баллом, а самым слабым звеном: нельзя открывать приложение, обрабатывающее данные о здоровье, с политикой конфиденциальности, в которой не назван получатель этих данных и не указано ответственное лицо.

24. Можно ли выпускать EatAps сегодня?

НЕТ — не для широкой публики в ЕС

Для закрытого теста на ограниченном круге согласившихся пользователей — да, с оговорками ниже.

Почему нет

- Политика конфиденциальности показывается людям как незаполненный шаблон: вместо имени и адреса ответственного стоят [Name] и [Anschrift], а сверху висит красная плашка «Проверьте у юриста». Документ в таком виде не является политикой.
- Приложение отправляет данные о здоровье — пол, возраст, вес, дневник питания и фотографии еды — в Anthropic, и нигде об этом не сообщает. Для особой категории данных по ст. 9 это самое серьёзное расхождение во всём отчёте.
- Stripe и Telegram тоже получают персональные данные и тоже не названы.
- Удаление аккаунта не удаляет загруженные фотографии: они остаются публично доступными по прямой ссылке навсегда. Человек нажал «удалить всё» — и это неправда.
- Любой пользователь может обойти дневной лимит AI параллельными запросами и тратить деньги владельца ключа. Это работает сегодня и требует скрипта в десять строк.
- Точка /api/feedback открыта без входа: личный Telegram владельца затапливается одной командой curl.
- На ветке лежит красный тест и расхождение setup_all.sql с миграциями — то есть установка базы с нуля даст не ту схему, что в проде.

Почему это всё равно хорошая новость

Ни одна из перечисленных проблем не является архитектурной. Переписывать ничего не нужно. Четырнадцать из двадцати двух находок закрываются правкой в несколько строк, а самая тяжёлая — удаление файлов из хранилища — это один-два дня работы.

Фундамент, на котором всё стоит, проверку выдержал: модель прав в базе выстроена верно и последовательно, подмены личности нет нигде, синхронизация между устройствами сделана на уровне, который редко встречается и в коммерческих продуктах. Дефекты здесь — на стыках (результат атомарной операции отброшен, файл пережил свою запись, документ не поспел за кодом), а не в основаниях.

Что обязательно сделать до запуска

№	Действие	ID	Оценка
1	Вернуть setup_all.sql к сборке и внести миграцию в SOURCES	DB-001	S
2	Закрыть гонку бюджета AI: сверять возвращённый итог с бюджетом	SEC-001	S
3	Потребовать вход в /api/feedback	SEC-003	S
4	Исправить TIER_BY_PRICE, чтобы отказ давал FREE, а не AI_PREMIUM	SEC-002	S
5	Дополнить Datenschutz, реестр ст. 30 и чек-лист AVV; заполнить плейсхолдеры	PRIV-001	M
6	Заключить договоры с Anthropic, Stripe и Telegram	PRIV-001	M
7	Удалять изображения при удалении записи, сообщения и аккаунта	PRIV-002	M
8	Закрыть бакеты post-images и chat-images по образцу dm-media	PRIV-002	L

№	Действие	ID	Оценка
9	Распространить блокировку на групповые диалоги	SEC-005	S
10	Добавить контроль порядка событий Stripe	SEC-004	M

Пункты 1–4 и 9 — это одна рабочая сессия. Пункты 5–6 требуют не программиста, а решения владельца и получаса в кабинетах Anthropic и Stripe. Пункты 7–8 и 10 — от одного до двух дней.

25. Приложение

25.1. Состав аудита

Коммит	6034515cde858d7c83a20bcd20a0092837d5a91
Ветка	cleanup/dead-is-favorite
Незакоммиченное	5 изменённых файлов + 1 новый (см. DB-001)
Node.js	v24.15.0
Платформа	macOS (darwin 25.0.0)

25.2. Выходные файлы

Файл	Содержание
EATAPS_FULL_AUDIT.pdf	Настоящий отчёт полностью
EATAPS_FULL_AUDIT.md	Тот же отчёт в Markdown
EATAPS_EXECUTIVE_SUMMARY.md	Краткая выжимка для владельца
EATAPS_FINDINGS.csv	22 находки со всеми полями
EATAPS_COVERAGE_MATRIX.csv	513 поэлементных строк покрытия

25.3. Как воспроизвести проверки

```
git rev-parse HEAD          # 6034515...
npm test                    # 813/814, падает sqlGuard → DB-001
npm run build               # успешно, предупреждение о чанках → PERF-001
npm run sql:verify          # падает → DB-001
node scripts/policy-audit.mjs # все политики по таблицам
node scripts/sql-idempotency.mjs # нарушений нет
npm audit                   # 17 записей → DEP-001

# SEC-002, проверка семантики отображения цены в тариф:
node -e "process.env.STRIPE_PRICE_AI='a';process.env.STRIPE_PRICE_AI_PLUS='b';
delete process.env.STRIPE_PRICE_AI_PREMIUM;
const M={ [process.env.STRIPE_PRICE_AI]:'AI',
[process.env.STRIPE_PRICE_AI_PLUS]:'AI_PLUS',
[process.env.STRIPE_PRICE_AI_PREMIUM]:'AI_PREMIUM'};
console.log(M[undefined])"      # → AI_PREMIUM
```

25.4. Оговорка о полноте

Этот отчёт описывает состояние одного коммита, полученное статическим разбором, запуском сборки и тестов и прогоном расчётных модулей на граничных данных. Он не заменяет ни проверку на живой базе от имени разных пользователей, ни браузерный прогон интерфейса, ни тест на проникновение, ни заключение юриста по правовым документам. Разделы 2.2 и 22 перечисляют границы прямо, и их следует читать вместе с любым выводом отчёта.

Находки со статусом PROBABLE (SEC-004 и UX-002) требуют подтверждения прежде, чем считать их

фактом. Все остальные подтверждены исполнением кода, выводом инструмента или однозначным чтением — и в этом смысле спорить о них можно только по приоритету, но не по существованию.